

Adaptive Hybrid RAG Interview Prep Guide

Two LangGraph agents · three retrieval modes · Corrective-RAG loop

Self-contained: includes the real implementation code.

LangGraph

Corrective RAG

Amazon Neptune / Gremlin

LMaaS · Azure OpenAI

IDAM OAuth token exchange

Vectorless retrieval

Knowledge graphs

SageMaker · VPC · PrivateLink

Mohammed Shadab Ameen

GE HealthCare · `hybrid-rag`

11 offline tests, 0.70 s, zero credentials · ~2,500 lines across 12 modules

This guide is self-contained. Every code block is the real implementation, quoted from the package, so the guide can be studied without access to the repository. Roughly 60% of the system's logic is reproduced here verbatim.

How to use it. §1–§2 is what you *say*. §3 is the code walkthrough — the part that matters if anyone asks "show me how you did X". §4–§6 are the inventory, the trade-off bank, and the war stories. §7–§9 is the question bank. §12 is the 20-minute pre-interview skim.

Contents

§	Section
1	Elevator pitches
2	System at a glance — diagrams, numbers, thresholds
3	Code walkthrough — module by module, with what to say
4	Skills & tools inventory
5	Alternatives not used, and why
6	Difficulties — real and anticipated
7	Q&A: architecture, RAG & CRAG
8	Q&A: LangGraph, LLM engineering & knowledge graphs
9	Q&A: Neptune, AWS, auth, security, testing & scale
10	Behavioral / STAR stories
11	Killer questions + questions to ask them
12	The 20-minute pre-interview skim

1. Elevator pitches

1.1 Ten seconds

"I built a retrieval system for medical-device service documentation that decides *for itself* how each document should be stored — as a knowledge graph or as embeddings — and then grades its own retrieval before it answers, so it corrects bad retrieval instead of hallucinating over it."

1.2 Thirty seconds — memorize this one

"Plain RAG has two failure modes that matter for service manuals: relationship questions don't survive chunking, and retrieval failure is silent — a vector store always returns its top-k even when nothing relevant exists, and the model then answers confidently over garbage.

So I built a two-agent system on **LangGraph**. **Agent 1** is an ingest router: it scans a whole document with a cheap LLM, counts distinct entities, and routes entity-dense documents into a **knowledge graph on Amazon Neptune** and narrative prose into the **vector store**, recording where everything went in a routing catalog. **Agent 2** is a **Corrective-RAG loop**: it uses that catalog to route the query, retrieves, then has a cheap LLM **grade its own retrieval** 0-to-1. Above 0.7 it answers. Below 0.3 it throws the retrieval away and recovers by having the LLM **reason over the document's table of contents** — vectorless retrieval, no embeddings at all. In between, it keeps both and fuses them.

Every model call runs on **LMaaS**, GE HealthCare's internal Azure OpenAI gateway, behind IDAM OAuth — nothing leaves the VPC. A typical query is 2–4 LLM calls, and only one uses the expensive model."

1.3 Two minutes — when they say "walk me through it"

Add, in order:

1. **The four failure modes it targets** (\$2.1). Lead with the problem, not the architecture.
2. **The two agents and three retrieval modes** — the 30-second pitch.
3. **The deliberate deviation from the paper**. "Canonical CRAG falls back to **web search** when retrieval grades poorly. I couldn't — no VPC egress — and I'd argue against it anyway: for internal device service documentation, answering from the open web is a *worse* failure than abstaining. So I kept the CRAG mechanism and swapped the corrective *source* for internal vectorless navigation."
4. **What's real vs. stubbed** — volunteer this. "LMaaS and Neptune are live: I write a real LLM-built typed graph into a real cluster from SageMaker over PrivateLink. The managed vector platform's exact API paths were still pending, so its client is implemented against the documented contract and the demo runs an in-memory stand-in. That's why everything is dependency-injected — 11 offline tests in 0.7 seconds with no credentials, and the real backends run the identical code paths."
5. **One hard-won detail** — the Neptune untyped-GraphSON header (\$6.4) or the `awselb` 403 (\$6.3).
6. **The honest limitation** — "the typed entity graph is persisted but not yet *traversed* at query time; the graph leg is still keyword search over chunk text. Multi-hop traversal is the clearest unclaimed payoff and I know exactly where it goes."

1.4 The deep version — for a staff/principal interviewer

"It's an adaptive hybrid RAG with a per-document storage policy and a per-query retrieval policy, joined by a routing catalog. Ingest treats 'does this document deserve a graph?' as an **evidence question, not a policy question**: windowed whole-document entity scan on the cheap deployment, windows sampled evenly if the document exceeds the call cap, entities normalized and unioned across windows, gate at ≥ 8 distinct entities. Entity-dense → Neptune only; prose → vector only. Mutually exclusive by default with a dual-store toggle, because duplicating entity-rich content as embeddings buys cost without capability.

Query is CRAG with the corrective source swapped for **vectorless navigation** — pull section headings only, have the cheap model pick 1–4 sections by meaning, fetch just those bodies. No embeddings, interpretable, and it specifically recovers the

failure mode where embedding coverage under-represents deep procedural content.

Cost discipline is structural rather than by cutting quality checks: two configured deployments and a `cheap` flag set at every call site — grading, navigation, refinement, classification, entity scan are all cheap; only final generation is strong. Plus early exit on a good grade, conditional refinement below a context-size threshold, and a zero-LLM-call shortcut that grades empty retrieval as incorrect without spending a call.

Both agents are compiled LangGraph `StateGraph`s with typed state and conditional edges, so control flow is deterministic and only the nodes are probabilistic — and every branch that fired comes back in a trace, so the loop is demonstrable rather than a black box."

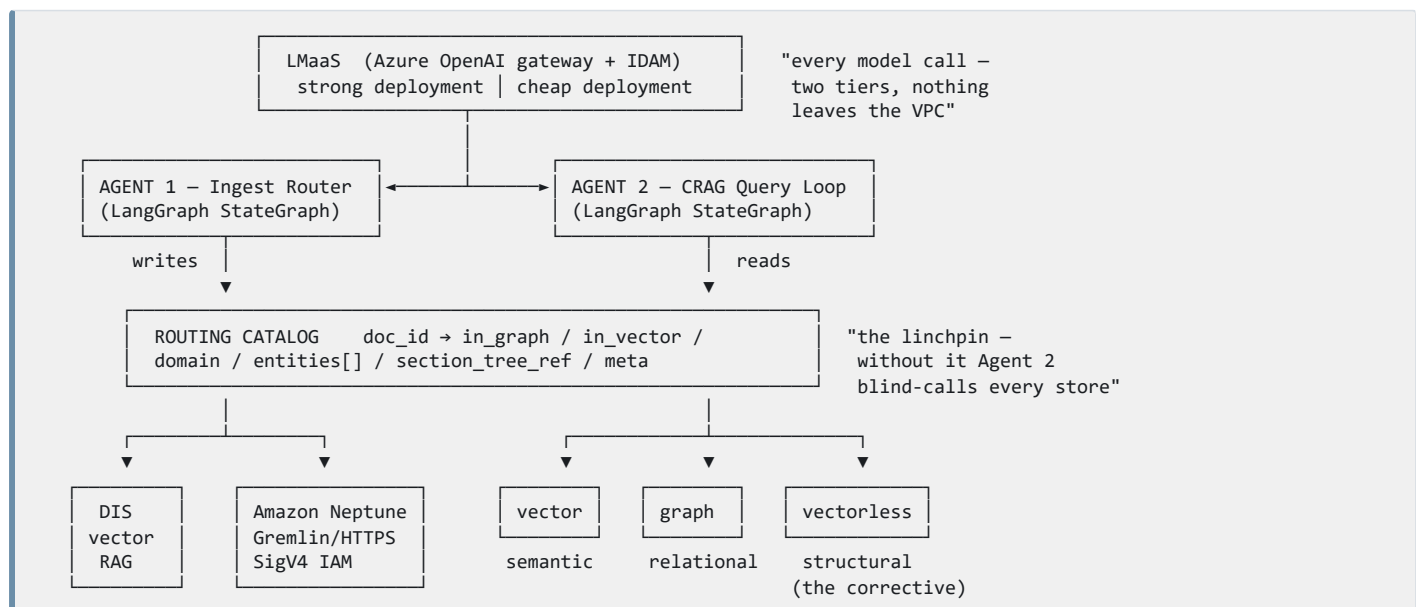
2. System at a glance

2.1 The four failure modes that justify the whole design

Lead with these. The architecture is only defensible as an answer to them.

1. **Relationship questions don't survive chunking.** "What causes error code E-204, and what procedure clears it?" is a **join across entities**, not a nearest-neighbour lookup. Embeddings retrieve passages that look similar; they cannot traverse `ErrorCode --INDICATES--> Fault --RESOLVED_BY--> Procedure`.
2. **Retrieval failure is silent.** A vector store always returns its top-k, even when nothing relevant exists. The generator then produces a confident answer over irrelevant context, and nothing in the pipeline says "this retrieval was bad."
3. **Not every document deserves a graph.** Building an entity graph over narrative prose costs LLM calls and yields a near-empty graph. The decision should be **per document and evidence-based**.
4. **Embedding coverage is uneven.** Deeply nested sections are frequently under-represented — so exactly the specific, procedural content a field engineer needs is what the vector store misses.

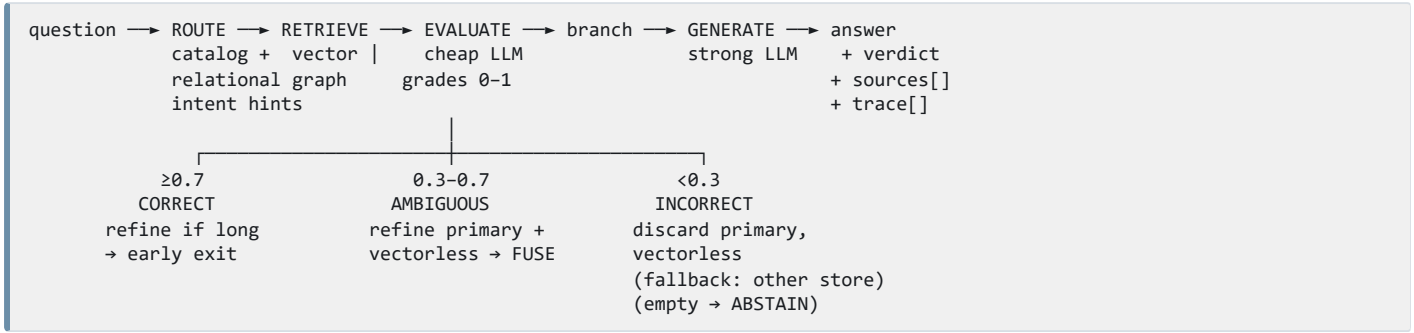
2.2 Whiteboard this in 90 seconds



Agent 1 — ingest:

```
document → ASSESS → distinct entities ≥ 8 ?
                    (windowed whole-doc scan, cheap LLM
                     + domain classification, cheap LLM)
                    |
                    +-- yes --> STORE_GRAPH (Neptune)
                    |           · Document→Section→Chunk
                    |           spine (deterministic)
                    |           · Entity nodes + MENTIONS
                    |           edges (LLM entities)
                    |
                    +-- no --> STORE_VECTOR (DIS)
                               · whole doc → collection
                    |
                    +--> WRITE_CATALOG
```

Agent 2 — query:



Three sentences to say while pointing at the branches: 1. "Correct is the cheap common path — one grade, maybe a refine, one generation." 2. "Ambiguous **widens** rather than replaces — the generator sees primary *and* vectorless. Fusion, not selection." 3. "Incorrect discards and corrects internally; if even that comes back empty it **abstains**, rather than inviting the generator to invent something."

2.3 The two graph layers — be precise, interviewers probe this

Layer	Nodes	Edges	Built by
Structural spine	Document, Section, Chunk	HAS_SECTION, HAS_CHUNK	Deterministic — markdown # or dotted 2.4.3 headings; ~120-word chunks
Entity layer on the spine	Entity (name-derived global id)	MENTIONS (Section → Entity)	LLM entities from the ingest scan, matched to section text
Typed semantic graph	one vertex per entity, label = LLM-assigned type (Component, ErrorCode, Procedure, Role), with name + etype	one typed edge per triple (CAUSES, INDICATES, PART_OF, RESOLVED_BY, POSITIONS) carrying occurrence count	LLM — subject/relation/object triples, windowed

Why split it that way: "Document structure is a parsing problem with a correct answer — spending LLM calls on it adds cost and variance for nothing. Relationship extraction is a judgement problem, which is what the model is good at. So I spend model calls only where there's genuine ambiguity."

2.4 Numbers to have on the tip of your tongue

Fact	Number
Offline test suite	11 tests, 0.70 s, zero credentials verified
Package size	~2,500 lines across 12 modules + demo + tests
LLM calls per query	2–4, exactly one on the strong model
Ingest calls per document	≤8 cheap entity-scan + 1 cheap domain + ≤6 strong triple + 1 strong PageIndex
LMaaS client surface	2 methods (complete_text, complete_json) — that's what makes the fakes possible
Demo corpus	3 synthetic documents (imaging service manual, field service handbook, reference manual)

2.5 Every threshold, and how to defend the number

Setting	Default	Meaning	How you defend it
Crag_Correct_Threshold	0.7	grade ≥ → trust retrieval, early exit	"Skepticism dial. Raise toward 0.8 → more corrections, higher cost, higher recall. Uncalibrated — I'd fit it against a labeled set, per corpus."

Setting	Default	Meaning	How you defend it
<code>CRAG_INCORRECT_THRESHOLD</code>	0.3	grade < → discard retrieval, correct	"Discard dial. Lower toward 0.2 → fuse more, discard less. The band between is deliberately wide because ambiguity is the common case."
<code>INGEST_MIN_GRAPH_ENTITIES</code>	8	distinct whole- doc entities to earn a graph	"Chosen so the number <i>means</i> what it says. The earlier gate counted ≤5 entities from the first 2000 chars against a threshold of 3 — it measured sample size, not the document (§6.2)."
<code>INGEST_ENTITY_SCAN_WINDOW_WORDS</code>	400	words per scan call	Small enough for a cheap model to enumerate entities reliably.
<code>INGEST_ENTITY_SCAN_MAX_WINDOWS</code>	8	hard cap on scan calls/doc	"Cost cap. Above it, windows are sampled evenly across the document , so coverage stays whole-document while calls stay bounded."
<code>REFINE_MIN_TOKENS</code>	180	skip refine below this context size	"Don't spend an LLM call to clean up two sentences."
<code>RETRIEVE_TOP_K</code>	5	hits per retriever	Conventional; the grader is the real quality gate.
<code>STORE_GRAPH_DOCS_IN_VECTOR</code>	false	dual-store toggle	"The routing decision is reversible by config — that's how you de-risk an opinionated default."
<code>LMAAS_REASONING_MODEL</code>	true	switches the request contract (\$6.5)	GPT-5.x needs <code>max_completion_tokens</code> and no <code>temperature</code> ; flip false for gpt-4o-class.
<code>NEPTUNE_USE_IAM</code>	true	SigV4-sign every Gremlin request	No DB passwords; access is IAM policy, auditable in CloudTrail.

3. Code walkthrough

Twelve modules, ~2,500 lines. This section quotes the load-bearing parts and — after each — the sentence to say about it. If an interviewer asks "how did you actually do that?", the answer is here.

```
hybridrag/
  config.py      all settings + thresholds; REPLACE_ME for every secret
  common.py      dataclasses (RetrievalResult, Verdict, CatalogEntry, QueryResult) + entity_id
  clients.py     LMaaSClient · DISClient · GraphClient + two IDAM token generators
  catalog.py     RoutingCatalog — the ingest→query contract
  ingest.py      AGENT 1 — StateGraph: assess → (route) → store_graph|store_vector → write_catalog
  retrieval.py   vector_retrieve · graph_retrieve · vectorless_retrieve
  crag.py        AGENT 2 — StateGraph: route → retrieve → evaluate → (branch) → generate
  kg_extract.py  LLM triple extraction + typed-graph assembly; LLM PageIndex extraction
  loader.py      load .md/.txt/.pdf — PDF headings detected by TYPOGRAPHY
  visualize.py   self-contained vis-network HTML (entity KG, PageIndex tree, document graph)
  fakes.py       FakeLMaaS / FakeDIS / FakeGraph — offline stand-ins
  demo.py        end-to-end offline demo
  tests/         11 offline orchestration tests
```

3.1 `common.py` — the contracts, and the one function that makes the graph connect

```
def entity_id(name: str) -> str:
    """Global (doc-independent) node id for an entity.

    The id is derived only from the entity NAME, so the same entity mentioned
    in different sections or different documents collapses to ONE graph node —
    that shared node is what turns the graph from a star into a web.
    """
    slug = re.sub(r"^[a-z0-9]+", "_", str(name).strip().lower()).strip("_")
    return f"ent_{slug}" if slug else "ent_unknown"

@dataclass
class RetrievalResult:
    """What any retriever (vector / graph / vectorless) returns."""
    source: str = "vector" # "vector" | "graph" | "vectorless"
    text: str # concatenated context for the generator
    chunks: list[dict] = field(default_factory=list) # [{doc_id, text, score, ...}]
    meta: dict = field(default_factory=dict)

    def is_empty(self) -> bool:
        return not (self.text and self.text.strip())

@dataclass
class Verdict:
    """CRAG retrieval-evaluator output."""
    label: str = "correct" # "correct" | "ambiguous" | "incorrect"
    score: float = 0.1 # 0..1 relevance of retrieval to the query
    reason: str = ""
```

Say this. "Two design decisions live in eight lines. `entity_id` derives the node id from the **name only** — not from the document — so the same entity in two windows or two manuals collapses to one shared node, and that shared node is the entire mechanism by which a set of extracted triples becomes a connected graph rather than a pile of stars. And `RetrievalResult` is a **uniform contract across all three retrieval modes**, so the CRAG loop treats vector, graph, and vectorless identically — `is_empty()` in particular is the trigger for the zero-cost 'incorrect' shortcut."

`QueryResult` is what Agent 2 returns, and the `trace` field is why the loop is demonstrable:


```
@dataclass
class QueryResult:
    query: str
    answer: str
    verdict: Verdict
    sources: list[str] = field(default_factory=list)  # which retrievers contributed
    llm_calls: int = 0
    trace: list[str] = field(default_factory=list)  # human-readable step log
```

Say this. "`llm_calls` and `trace` are first-class return values, not logging. I wanted cost and the branch that fired to be **observable from the outside** — that's what turns a demo from 'trust me' into 'here's which path it took and what it cost'."

3.2 `config.py` — thresholds and the fail-fast guard

```
# Retrieval-evaluator thresholds (score in [0,1]): the CRAG verdict boundaries.
CRAG_CORRECT_THRESHOLD: float = float(os.environ.get("CRAG_CORRECT_THRESHOLD", "0.7"))
CRAG_INCORRECT_THRESHOLD: float = float(os.environ.get("CRAG_INCORRECT_THRESHOLD", "0.3"))

# Only run the (extra) refinement call when context is bigger than this (tokens ~ words).
REFINE_MIN_TOKENS: int = int(os.environ.get("REFINE_MIN_TOKENS", "180"))

# --- Agent-1 ingest: full-document graph-worthiness gate -----
# Agent 1 scans the WHOLE document (windowed, cheap model), counts the DISTINCT
# entities, and builds a graph only if there are enough to make relationships
# worthwhile. This replaces the old "count a 2000-char sample" heuristic.
INGEST_MIN_GRAPH_ENTITIES: int = int(os.environ.get("INGEST_MIN_GRAPH_ENTITIES", "8"))
# Words per entity-scan window (each window = one cheap LLM extraction call).
INGEST_ENTITY_SCAN_WINDOW_WORDS: int = int(os.environ.get("INGEST_ENTITY_SCAN_WINDOW_WORDS", "400"))
# Cost cap: never make more than this many scan calls per document. If the doc
# has more windows than this, they are sampled EVENLY across it (so we still
# "look at the whole document", just not every window on very long docs).
INGEST_ENTITY_SCAN_MAX_WINDOWS: int = int(os.environ.get("INGEST_ENTITY_SCAN_MAX_WINDOWS", "8"))

# Storage-routing policy. By default an entity-dense document goes to the GRAPH
# ONLY (not also the vector store) – the graph + vectorless retrieval serve it,
# and we avoid duplicating entity-rich content as vectors.
STORE_GRAPH_DOCS_IN_VECTOR: bool = (
    os.environ.get("STORE_GRAPH_DOCS_IN_VECTOR", "false").lower() == "true"
)

def is_configured(value: str) -> bool:
    """True if a setting has been filled in (not left as REPLACE_ME/empty)."""
    return bool(value) and value != "REPLACE_ME"
```

Say this. "Every secret and endpoint defaults to the literal string `REPLACE_ME`, and `is_configured` is what the real clients check before doing anything. So an unconfigured system **tells you exactly which settings are missing** instead of surfacing a confusing 401 from a service. Small thing, saved a lot of debugging. And every threshold is env-overridable because I expect the right values to be **corpus-specific** — they belong in config, not in the code."

3.3 `catalog.py` — the ingest→query contract

The catalog is the only coupling between the two agents. Two methods do the query-time work:

```

def candidates(self, query: str) -> dict[str, bool]:
    """Which stores plausibly hold docs relevant to the query.

    Prototype heuristic: overlap the query terms with catalogued entities /
    domains; if nothing matches, fall back to whatever stores exist. Keeps
    Agent 2 from calling a store that has no relevant content.
    """
    q = query.lower()
    vector = graph = False
    matched = False
    for e in self._entries.values():
        hay = " ".join([e.title, e.domain, " ".join(e.entities)]).lower()
        if any(tok for tok in _terms(q) if tok in hay):
            matched = True
            vector = vector or e.in_vector
            graph = graph or e.in_graph
    if not matched: # no strong signal -> allow any store that exists
        vector, graph = self.has_vector(), self.has_graph()
    return {"vector": vector, "graph": graph}

def candidate_doc_ids(self, query: str, *, in_graph: bool = True) -> list[str]:
    """doc_ids whose title/domain/entities overlap the query.

    Used to SCOPE vectorless navigation to the relevant documents (so it reads
    a focused table-of-contents, not the whole corpus). Falls back to every
    graph doc when nothing matches, so vectorless still has something to read.
    """
    q_terms = _terms(query.lower())
    pool = [e for e in self._entries.values() if (e.in_graph or not in_graph)]
    matched = []
    for e in pool:
        hay = " ".join([e.title, e.domain, " ".join(e.entities)]).lower()
        if any(t in hay for t in q_terms):
            matched.append(e.doc_id)
    return matched or [e.doc_id for e in pool]

def _terms(text: str) -> list[str]:
    return [t for t in text.replace(" ", "").replace("?", "").split() if len(t) >= 4]

```

Say this. "Two questions, two methods. `candidates` answers *which stores* could hold relevant content — that's what stops Agent 2 blind-calling a store with nothing in it. `candidate_doc_ids` answers *which documents*, and that's what keeps vectorless navigation cheap as the corpus grows: the LLM reads a focused table of contents from the relevant documents, not the whole corpus.

Note both **fall back rather than starve** — no match means 'allow any store that exists' and 'every graph doc', so a query is never left with no route. And I'll name the weakness before you do: it's bag-of-words overlap with a length-4 filter. No stemming, no synonyms, no semantics. The upgrade is **entity-anchored routing** — resolve query mentions to actual entity nodes and route by graph neighbourhood. What saves it today is that the grader catches a bad primary, so routing being wrong is a recoverable event rather than a failure."

3.4 `ingest.py` — Agent 1

The prompts

```

_DOMAIN_SYSTEM = """Assign a short domain label to the document. Return ONLY JSON:
{"domain": "<short label, e.g. service-manual, policy, faq, spec>"}
"""

_ENTITY_SYSTEM = """List every DISTINCT salient entity in the text – named components, features,
tools, procedures, parameters, error codes, roles. Return ONLY JSON:
{"entities": [<canonical name>, "..."]}
Rules:
- Canonical names only; do NOT list the same entity twice.
- Ignore document furniture (page numbers, headers/footers, generic filler).
- Return an empty list if the text names nothing of substance.
"""

```

Say this. "Three things a beginner leaves out. **Canonical names** — because entity merging is by name, so the prompt is doing data-modelling work, not just extraction. **Explicit furniture exclusion** — real PDF text is full of page numbers and footers and they silently poison extraction. And **permission to return nothing** — without that line, a model asked for entities will manufacture them."

Whole-document windowing with even sampling

```
def _windows(text: str, window_words: int, max_windows: int) -> list[str]:
    """Split the FULL text into word-windows; if there are more than max_windows,
    sample them EVENLY across the document so coverage stays whole-document while
    the number of LLM calls stays bounded."""
    words = text.split()
    if not words:
        return []
    wins = [" ".join(words[i:i + window_words]) for i in range(0, len(words), window_words)]
    if len(wins) <= max_windows:
        return wins
    idxs = sorted({round(i * (len(wins) - 1) / (max_windows - 1)) for i in range(max_windows)})
    return [wins[i] for i in idxs]
```

Say this. "The `idxs` line is the whole idea: rather than truncating to the first 8 windows — which would assess only the front of a long manual — it spreads 8 sample points **evenly from start to end**. Coverage stays whole-document; cost stays bounded at 8 calls. The `sorted({...})` set collapses duplicate indices on short documents so I never scan the same window twice."

The graph-worthiness gate

```
def assess_document(self, title: str, text: str) -> dict:
    """Scan the WHOLE document and decide graph-worthiness by distinct-entity count.

    Returns {domain, entities, entity_count, word_count, density, build_graph}.
    """
    windows = _windows(text, config.INGEST_ENTITY_SCAN_WINDOW_WORDS,
                       config.INGEST_ENTITY_SCAN_MAX_WINDOWS)
    seen: dict[str, str] = {} # normalized -> display form (dedupe across windows)
    for w in windows:
        try:
            out = self.lmaas.complete_json(_ENTITY_SYSTEM, w, cheap=True)
        except Exception as exc: # noqa: BLE001 - a bad window shouldn't abort ingest
            logger.warning("entity scan window failed (%s)", exc)
            continue
        for e in (out.get("entities") if isinstance(out, dict) else []) or []:
            key = _norm_entity(e)
            if key and key not in seen:
                seen[key] = str(e).strip()

    entities = list(seen.values())
    word_count = len(text.split())
    entity_count = len(entities)
    density = round(entity_count / max(1.0, word_count / 1000.0), 2) # entities per 1k words
    build_graph = entity_count >= self.min_graph_entities
    logger.info("assessed %s: %d distinct entities across %d words (density %.2f/1k) -> graph=%s",
               title, entity_count, word_count, density, build_graph)
    return {"domain": self._domain(title, text), "entities": entities,
            "entity_count": entity_count, "word_count": word_count,
            "density": density, "build_graph": build_graph}
```

Say this. "Four points. It's the **whole document**, windowed — not a sample of page one. Entities are **normalized and de-duplicated** across windows via the `seen` dict, so the count is genuinely distinct entities and not a sum of per-window lists. A **failed window is logged and skipped, not fatal** — one bad JSON response must not lose an entire manual. And `density` is recorded as a diagnostic but the **gate is on the absolute distinct count**, because that's the number a reader can reason about."

Heading detection — and why it's deliberately narrow

```
# Pure document furniture (footer codes / bare page numbers) – never a heading
# or chunk. Mirrors loader._is_boilerplate so .md/.txt get the same cleanup.
_BOILER_RE = re.compile(
    r"^(?:\d{1,4}|\d{3,}[A-Za-z]?s+EN\s+\d{4,}|\d{3,}[A-Za-z]?|page\s+\d+)$", re.I)

def _is_heading(line: str) -> bool:
    """Trust only strong heading signals: a markdown ``#`` (native to .md, and
    emitted by the PDF loader from font size) or dotted multi-level numbering
    (``2.4.3 Foo``). Deliberately NOT single-number list steps (``1 Do X``) or
    short ALL-CAPS lines (UI labels) – those flooded the old tree with noise.
    """
    s = line.strip()
    if not s:
        return False
    return s.startswith("#") or bool(re.match(r"^\d+\.\d+", s))
```

Say this — this is a good "tell me about an iteration" answer. "The first version treated single-number lines (`1 Do X`) and short ALL-CAPS lines as headings. On real manual text that **flooded the section tree with noise** — every numbered procedure step and every UI label became a section, which wrecked vectorless navigation because the table of contents was garbage. So I narrowed it to two strong signals only: a markdown `#`, or dotted multi-level numbering.

That created a follow-on problem — real PDFs don't have `#`. Which is why the **PDF loader detects headings by typography and emits** `#`, so structure detection lives where the font information is and the section parser stays trivially simple. Two modules, one contract."

The section tree and entity linking

```
def _naive_sections(text: str) -> list[dict]:
    """Split structured text into a section tree (heading + chunks).

    Relies on :func:`_is_heading`; boilerplate lines are dropped outright. A
    heading with no body (e.g. a chapter title page) is kept as a structural
    node, but a body-less leading "Introduction" placeholder is discarded.
    """
    sections: list[dict] = []
    cur = {"order": 0, "heading": "Introduction", "body": []}
    for raw in text.splitlines():
        s = raw.strip()
        if not s or _is_boilerplate(s):
            continue
        if _is_heading(s):
            if cur["body"] or cur["heading"] != "Introduction":
                sections.append(cur)
                heading = re.sub(r"^{1,6}\s*", "", s).strip()
                cur = {"order": len(sections), "heading": heading, "body": []}
            else:
                cur["body"].append(s)
        if cur["body"] or cur["heading"] != "Introduction":
            sections.append(cur)
    for s in sections:
        s["chunks"] = _chunk(" ".join(s.pop("body")))
    return sections or [{"order": 0, "heading": "Document", "chunks": _chunk(text)}]

def _entity_mentions(sections: list[dict], entities: list[str],
                     max_per_entity: int = 12) -> list[dict]:
    """Link each extracted entity to the sections whose text mentions it
    (case-insensitive substring). An entity that appears in several sections
    becomes a shared hub node – this is what interconnects the graph. Entities
    that match no section text are skipped (no orphan nodes).
    """
    haystacks = [(s.get("order", 0),
                    (s.get("heading", "") + " " + " ".join(s.get("chunks", []))).lower())
                  for s in sections]
    mentions: list[dict] = []
    for ent in entities or []:
        e = str(ent).strip()
        if len(e) < 3:
            continue
        el = e.lower()
        hits = 0
        for order, text in haystacks:
            if el in text:
                mentions.append({"order": order, "entity": e})
                hits += 1
                if hits >= max_per_entity:
                    break
    return mentions
```

Say this. "Three defensive choices worth naming. The final `or [...]` means a document with **no detectable headings still produces one section** — it degrades to flat chunking instead of returning an empty tree, so nothing silently vanishes. Entities that **match no section text are skipped**, so the graph never accumulates orphan nodes the LLM hallucinated. And `max_per_entity` caps mention edges at 12 per entity, because a term like 'Viewport' appearing in 200 sections produces 200 useless edges and a hairball visualization."

The router and the deterministic flag computation

```
def _route_store(self, state: IngestState) -> str:
    # entity-dense -> graph path; otherwise -> vector path
    return "graph" if state["assessment"]["build_graph"] else "vector"

def _n_store_graph(self, state: IngestState) -> IngestState:
    sections = _naive_sections(state["text"])
    self.graph.upsert_document_tree(state["doc_id"], state["title"], sections)
    # Link the entities found during assessment to the sections that mention
    # them, so the graph gets Entity nodes + cross-references (interconnections)
    # instead of a bare Document->Section->Chunk star.
    mentions = _entity_mentions(sections, state["assessment"]["entities"])
    if mentions:
        self.graph.upsert_entities(state["doc_id"], mentions)
    # entity-dense docs are graph-only by default; optionally dual-store.
    if config.STORE_GRAPH_DOCS_IN_VECTOR:
        self._upload_vector(state)
    return {}

def _n_write_catalog(self, state: IngestState) -> IngestState:
    a = state["assessment"]
    # Flags are a deterministic function of the routing decision + config,
    # so compute them here rather than relying on cross-node state merges.
    build_graph = a["build_graph"]
    in_graph = build_graph
    in_vector = (not build_graph) or config.STORE_GRAPH_DOCS_IN_VECTOR
    entry = CatalogEntry(
        doc_id=state["doc_id"], title=state["title"], in_vector=in_vector, in_graph=in_graph,
        collection=self.collection, domain=a["domain"],
        entities=a["entities"][:40],
        section_tree_ref="neptune" if in_graph else "",
        meta={"entity_count": a["entity_count"], "density": a["density"],
            "word_count": a["word_count"], "min_graph_entities": self.min_graph_entities},
    )
    self.catalog.put(entry)
    return {"entry": entry}
```

Say this — this is your LangGraph war story (\$6.6). "That comment is load-bearing. I originally propagated `in_graph` / `in_vector` as node state from the two branch nodes, and **cross-branch state merges after a conditional edge proved unreliable** — a value set in one branch wasn't dependably visible after the merge. Rather than fight the framework, I compute the flags **deterministically** from `build_graph` plus the dual-store config, in one place. Same result, no dependence on a guarantee I couldn't verify. I documented it so nobody 'fixes' it back."

Also note `meta` records `min_graph_entities` alongside the count — "so a catalog entry is **self-explaining**: you can see not just that it went to the graph, but the bar it cleared and by how much. That matters when you change the threshold later and want to know which documents would re-route."

The StateGraph

```
def _build_app(self) -> Any:
    from langgraph.graph import END, START, StateGraph

    g = StateGraph(IngestState)
    g.add_node("assess", self._n_assess)
    g.add_node("store_vector", self._n_store_vector)
    g.add_node("store_graph", self._n_store_graph)
    g.add_node("write_catalog", self._n_write_catalog)

    g.add_edge(START, "assess")
    # the router: entity-dense docs take the graph path, others the vector path
    g.add_conditional_edges("assess", self._route_store,
        {"graph": "store_graph", "vector": "store_vector"})
    g.add_edge("store_graph", "write_catalog")
    g.add_edge("store_vector", "write_catalog")
    g.add_edge("write_catalog", END)
    return g.compile()
```

Say this. "Compiled once in `__init__`, invoked per document. Note `assess_document` is **reusable standalone logic** and `_n_assess` is a three-line wrapper — so the routing decision is unit-testable without constructing a graph at all. That's the pattern throughout: the interesting logic is pure functions; the StateGraph is wiring."

3.5 `retrieval.py` — three modes, one contract

```
def vector_retrieve(dis, query: str, collection: str, top_k: int | None = None) -> RetrievalResult:
    top_k = top_k or config.RETRIEVE_TOP_K
    passages = dis.retrieve(query, collection, top_k=top_k) or []
    text = "\n\n".join(p.get("text", "") for p in passages if p.get("text"))
    return RetrievalResult(source="vector", text=text, chunks=passages,
                           meta={"collection": collection, "k": top_k})

def graph_retrieve(graph, query: str, limit: int | None = None) -> RetrievalResult:
    limit = limit or config.RETRIEVE_TOP_K
    hits = graph.keyword_search(query, limit=limit) or []
    text = "\n\n".join(h.get("text", "") for h in hits if h.get("text"))
    return RetrievalResult(source="graph", text=text, chunks=hits, meta={"limit": limit})
```

Vectorless — the corrective, and the most interesting 30 lines in the project

```
_NAV_SYSTEM = """You navigate a document by its table of contents. Given a question and a
list of section headings (each with an id), pick the section ids most likely to contain the
answer — reason about meaning, not keywords. Return ONLY JSON:
{"section_ids": ["<id>", "..."]}
Rules:
- Choose few, high-precision sections (usually 1-4). Empty list if none look relevant.
"""

def vectorless_retrieve(lmaas, graph, query: str, doc_ids: list[str] | None = None,
                       max_sections: int = 4) -> RetrievalResult:
    """Reasoning-based retrieval over the section tree (no embeddings).

    1) pull headings (tiny), 2) LLM picks relevant section ids, 3) fetch their text.
    """
    sections = graph.get_sections(doc_ids) or []
    if not sections:
        return RetrievalResult(source="vectorless", text="", chunks=[])

    toc = "\n".join(f"- {s['section_id']}: {s.get('heading', '')}" for s in sections)
    try:
        out = lmaas.complete_json(_NAV_SYSTEM, f"QUESTION: {query}\n\nHEADINGS:\n{toc}", cheap=True)
    except Exception as exc:
        # noqa: BLE001 — fall back to no selection
        logger.warning("vectorless navigation failed (%s)", exc)
        out = {}

    chosen = (out.get("section_ids") if isinstance(out, dict) else None) or []
    chosen = chosen[:max_sections]
    if not chosen:
        return RetrievalResult(source="vectorless", text="", chunks=[],
                              meta={"navigated": True, "chosen": []})

    texts = graph.get_section_texts(chosen) or []
    chunks = []
    for t in texts:
        body = t.get("text")
        body = " ".join(body) if isinstance(body, list) else (body or "")
        chunks.append({"section_id": t.get("section_id"), "heading": t.get("heading"), "text": body})
    text = "\n\n".join(f"## {c['heading']}\n{c['text']}" for c in chunks if c["text"])
    return RetrievalResult(source="vectorless", text=text, chunks=chunks,
                          meta={"navigated": True, "chosen": chosen})
```

Say this — rehearse it, it's your best technical moment. "Two Gremlin calls and one cheap LLM call. `get_sections` returns **headings only**, which is a tiny payload even for a 300-page manual. The model picks section ids by **meaning, not keyword match**, capped at four for precision. Then `get_section_texts` fetches **only the chosen bodies**. It's how a human uses a manual: read the contents, flip to the chapter.

Four properties make it earn its place. **No embeddings** — nothing to provision, nothing to re-index when a document changes. **Cheap on large documents**, because deciding costs headings, not text. **Interpretable** — `meta["chosen"]` records

exactly which sections it picked, so you can judge whether that was reasonable, which you cannot do with a similarity score. And it's strongest precisely where embeddings are weakest: **deep, specific, nested procedural sections**.

Note it returns an **empty result rather than raising** on navigation failure or an empty selection — that's what lets the CRAG loop fall back further instead of dying. And `doc_ids` is how Agent 2 scopes it to catalog candidates, so the table of contents stays focused as the corpus grows."

3.6 `crag.py` — Agent 2

The three prompts

```
_EVAL_SYSTEM = """You judge whether retrieved context can answer a question. Return ONLY JSON:
{"score": <float 0..1>, "reason": "<short>"}
- 1.0 = fully answers; 0.0 = irrelevant/empty. Judge relevance & sufficiency, not style.
"""

_REFINE_SYSTEM = """You clean retrieved context for a question. Decompose the context into facts,
DROP anything irrelevant to the question, and recompose the kept facts into tight prose.
Return ONLY the cleaned text (no preamble). Keep only what helps answer the question.
"""

_GEN_SYSTEM = """You answer the question using ONLY the provided knowledge. Be accurate and concise.
If the knowledge is insufficient, say what is missing. Do not invent facts.
"""
```

Say this. "Note what each one *excludes*. The evaluator is told to judge **relevance and sufficiency, not style** — otherwise a well-written irrelevant passage scores high. Refine is **decompose** → **filter** → **recompose**, which is the CRAG paper's formulation, and 'no preamble' because a chatty model wraps output in 'Here is the cleaned context:'. And the generator gets two constraints that matter more than 'be accurate': **use only the provided knowledge**, and **say what is missing** — giving the model an approved way to fail is what stops it inventing."

The evaluator — including both cost shortcuts

```
def evaluate(lmaas, query: str, result: RetrievalResult) -> Verdict:
    if result.is_empty():
        return Verdict(label="incorrect", score=0.0, reason="no context retrieved")
    try:
        out = lmaas.complete_json(
            _EVAL_SYSTEM, f"QUESTION: {query}\n\nCONTEXT:\n{result.text[:4000]}", cheap=True)
        score = float(out.get("score", 0.0)) if isinstance(out, dict) else 0.0
        reason = out.get("reason", "") if isinstance(out, dict) else ""
    except Exception as exc: # noqa: BLE001 - evaluator failure -> ambiguous (safe)
        logger.warning("evaluator failed (%s)", exc)
        return Verdict(label="ambiguous", score=0.5, reason="evaluator error")
    score = max(0.0, min(1.0, score))
    if score >= config.CRAG_CORRECT_THRESHOLD:
        label = "correct"
    elif score < config.CRAG_INCORRECT_THRESHOLD:
        label = "incorrect"
    else:
        label = "ambiguous"
    return Verdict(label=label, score=score, reason=reason)
```

Say this. "Four deliberate behaviours in twenty lines. **Empty retrieval short-circuits to incorrect with zero LLM calls** — grading nothing is pure waste. Context is **clipped at 4000 characters**, because the grader needs a sample to judge relevance, not the whole context. The score is **clamped to [0,1]** because a model asked for a float will occasionally return 1.5. And the failure mode is the important one: **an evaluator error defaults to ambiguous, not to trust or discard** — so a grader failure *widens* retrieval. Fail toward more evidence."

Conditional refinement and fusion

```
def refine(lmaas, query: str, text: str) -> tuple[str, bool]:
    """Returns (refined_text, called). Skips the LLM call for small context."""
    if len(text.split()) < config.REFINE_MIN_TOKENS:
        return text, False
    try:
        return lmaas.complete_text(_REFINE_SYSTEM, f"QUESTION: {query}\n\nCONTEXT:\n{text}",
                                   cheap=True, max_tokens=1024), True
    except Exception as exc: # noqa: BLE001
        logger.warning("refine failed (%s); using raw context", exc)
        return text, False

def generate(lmaas, query: str, knowledge: str) -> str:
    return lmaas.complete_text(
        _GEN_SYSTEM, f"QUESTION: {query}\n\nKNOWLEDGE:\n{knowledge}", cheap=False, max_tokens=1024)

def _fuse(k_in: str, k_ex: str) -> str:
    return "\n\n---\n\n".join(p for p in (k_in, k_ex) if p and p.strip())
```

Say this. " `refine` returns a `(text, called)` tuple so the call counter stays honest — I want the reported cost to be the real cost, not an estimate. It also **never fails the query**: on error it returns the raw context. And `generate` is the **only** place in the entire system that passes `cheap=False`. That single flag is the whole cost strategy, enforced at the call site rather than in a config table.

`_fuse` is four lines and it's a design position: concatenate with a separator and let the strong generator arbitrate. I don't rank or merge, because the two knowledge blocks have **incomparable scores** — one's a retrieval similarity, the other's an LLM section choice. Inventing a shared scale would be pretending to knowledge I don't have."

Query routing

```
def _primary_source(self, query: str) -> str:
    cand = self.catalog.candidates(query)
    relational = any(h in query.lower() for h in _RELATIONAL_HINTS)
    if relational and cand.get("graph"):
        return "graph"
    if cand.get("vector"):
        return "vector"
    if cand.get("graph"):
        return "graph"
    return "vector"

_RELATIONAL_HINTS = ("relationship", "related", "depend", "connect", "between",
                     "how does", "linked", "cause", "impact")
```

Say this. "Catalog first, then intent, with an explicit precedence I can explain in one sentence: **relational-sounding questions prefer the graph; otherwise the store that actually holds the relevant document wins**; and the final `return "vector"` is the cold-catalog default so a query is never unrouted. The hint list is a keyword heuristic and I'd call it the crudest part of the system — but it's cheap, it's zero-latency, and being wrong is recoverable because the grader checks the result."

The three branch nodes — where the CRAG policy lives

```
def _n_correct(self, state: QueryState) -> QueryState:
    text, called = refine(self.lmaas, state["query"], state["primary"].text)
    return {"knowledge": text, "sources": [state["primary_source"]],
           "calls": state.get("calls", 0) + (1 if called else 0)}

def _n_ambiguous(self, state: QueryState) -> QueryState:
    k_in, called = refine(self.lmaas, state["query"], state["primary"].text)
    k_ex = self._vectorless(state["query"])
    calls = state.get("calls", 0) + (1 if called else 0) + 1 # +1 vectorless nav
    note = "empty" if k_ex.is_empty() else f"{len(k_ex.chunks)} sections"
    sources = [state["primary_source"]] + (["vectorless"] if not k_ex.is_empty() else [])
    return {"knowledge": _fuse(k_in, k_ex.text), "sources": sources, "calls": calls,
           "trace": state["trace"] + [f"corrective (vectorless): {note}"]}

def _n_incorrect(self, state: QueryState) -> QueryState:
    k_ex = self._vectorless(state["query"])
    calls = state.get("calls", 0) + 1
    trace = state["trace"] + [f"corrective (vectorless): "
                             f"{'empty' if k_ex.is_empty() else str(len(k_ex.chunks)) + ' sections'}"]
    if k_ex.is_empty():
        other = "graph" if state["primary_source"] == "vector" else "vector"
        alt = self._retrieve(other, state["query"])
        trace.append(f"fallback to {other}: {'empty' if alt.is_empty() else str(len(alt.chunks)) + ' chunks'}")
        return {"knowledge": alt.text, "sources": [other] if not alt.is_empty() else [],
               "calls": calls, "trace": trace}
    return {"knowledge": k_ex.text, "sources": ["vectorless"], "calls": calls, "trace": trace}

def _n_generate(self, state: QueryState) -> QueryState:
    knowledge = state.get("knowledge", "") or ""
    if not knowledge.strip():
        return {"answer": "I couldn't find enough internal information to answer this confidently.",
               "trace": state["trace"] + ["no knowledge assembled – abstaining"]}
    answer = generate(self.lmaas, state["query"], knowledge)
    return {"answer": answer, "calls": state.get("calls", 0) + 1}
```

Say this. "Read the three branches as three policies. **Correct** keeps the primary and exits — `sources` is just the primary, no correction attempted. **Ambiguous** keeps the primary *and* adds vectorless and fuses — widen, don't replace — and note `sources` only lists vectorless **if it actually returned something**, so the reported provenance is truthful rather than aspirational. **Incorrect** discards the primary entirely, and if vectorless is *also* empty it falls back to the **other internal store** — defense in depth for the case where both the router and the navigator whiff. Never the web.

And `_n_generate` is where abstention lives: if no knowledge was assembled, it returns a fixed message and **never calls the generator at all**. That's deliberate on two axes — no opportunity to confabulate, and no strong-model spend on a query we can't answer."

The StateGraph and the public API

```
def _build_app(self) -> Any:
    from langgraph.graph import END, START, StateGraph

    g = StateGraph(QueryState)
    g.add_node("route", self._n_route)
    g.add_node("retrieve", self._n_retrieve)
    g.add_node("evaluate", self._n_evaluate)
    g.add_node("correct", self._n_correct)
    g.add_node("ambiguous", self._n_ambiguous)
    g.add_node("incorrect", self._n_incorrect)
    g.add_node("generate", self._n_generate)

    g.add_edge(START, "route")
    g.add_edge("route", "retrieve")
    g.add_edge("retrieve", "evaluate")
    g.add_conditional_edges("evaluate", self._route_after_eval,
                           {"correct": "correct", "ambiguous": "ambiguous",
                            "incorrect": "incorrect"})
    for branch in ("correct", "ambiguous", "incorrect"):
        g.add_edge(branch, "generate")
    g.add_edge("generate", END)
    return g.compile()

def _route_after_eval(self, state: QueryState) -> str:
    return state["verdict"].label # "correct" | "ambiguous" | "incorrect"

def run(self, query: str) -> QueryResult:
    final = self.app.invoke({"query": query})
    verdict = final.get("verdict") or Verdict(label="incorrect", score=0.0)
    return QueryResult(query=query, answer=final.get("answer", ""), verdict=verdict,
                       sources=final.get("sources", []), llm_calls=final.get("calls", 0),
                       trace=final.get("trace", []))
```

Say this. "This is the clearest argument for LangGraph over plain functions. `_route_after_eval` is a **one-line pure function returning the verdict label**, and the conditional-edge map turns that label into a branch. My core design decision — three corrective policies selected by a grade — is **declared in the graph**, not buried in if-statements across a call stack. And `run` keeps the public API stable: whatever the internal state looks like, callers get a `QueryResult`. That's what let me convert this from plain functions to a StateGraph without touching a single test."

Cost, by path

Verdict	LLM calls	Models
correct	1 eval + (0–1 refine) + 1 gen = 2–3	cheap, cheap, strong
ambiguous	1 eval + (0–1 refine) + 1 nav + 1 gen = 3–4	cheap×N, strong
incorrect	1 eval + 1 nav + 1 gen = 3 (+1 if store fallback)	cheap×N, strong
empty retrieval → incorrect	0 eval + 1 nav + 1 gen = 2	cheap, strong
abstain	0 gen — no strong call at all	cheap only

3.7 `kg_extract.py` — the LLM builds the graph

The triple-extraction contract

```
_TRIPLES_SYSTEM = """You are building a knowledge graph from a technical document.
Extract the factual relationships it states as subject-relation-object triples.
Return ONLY JSON of this exact shape:
{"triples": [
  {"subject": "<entity>", "subject_type": "<Type>",
   "relation": "<RELATION>",
   "object": "<entity>", "object_type": "<Type>"}
]}
Rules:
- Entities are concrete named things: components, error codes, procedures, roles,
  systems, parameters. Use canonical names (e.g. "Error Code E-204", "Collimator").
- Types are short PascalCase labels you choose, e.g. Component, ErrorCode,
  Procedure, Role, System, Parameter.
- Relations are short UPPER_SNAKE_CASE verbs, e.g. CAUSES, INDICATES, PART_OF,
  RESOLVED_BY, PERFORMS, RUNS_ON, REQUIRES, POSITIONS.
- Only include relationships the text explicitly supports. Do NOT invent facts.
- Ignore document furniture (page numbers, bare headings, generic filler).
- Return an empty list if the text asserts no concrete relationships.
"""
```

A real extraction, for reference in conversation:

```
{"triples": [
  {"subject": "Error Code E-204", "subject_type": "ErrorCode",
   "relation": "INDICATES",
   "object": "Collimator Alignment Fault", "object_type": "Fault"},
  {"subject": "Error Code E-204", "subject_type": "ErrorCode",
   "relation": "RESOLVED_BY",
   "object": "Collimator Alignment Routine", "object_type": "Procedure"},
  {"subject": "Gantry", "subject_type": "Component",
   "relation": "POSITIONS",
   "object": "Detector", "object_type": "Component"}
]}
```

Say this. "The casing conventions aren't cosmetic — **PascalCase types and UPPER_SNAKE_CASE relations** make the downstream Gremlin-label sanitization predictable, so `"part of"` becoming `PART_OF` is the rule rather than a repair. And note the model **chooses its own type vocabulary**. That's the trade I made: no hand-written schema per domain, at the cost of label drift across a large corpus — which I'll name as a real risk and tell you how I'd fix it."

The assembly rules — "the LLM proposes, code enforces"

```
def add_triple(self, t: dict, doc_id: str = "") -> bool:
    s, sty = _clean(t.get("subject")), _clean(t.get("subject_type")) or "Entity"
    o, oty = _clean(t.get("object")), _clean(t.get("object_type")) or "Entity"
    rel = _clean(t.get("relation")).upper().replace(" ", "_")
    if not s or not o or not rel or len(s) < 2 or len(o) < 2:
        return False
    sid, oid = entity_id(s), entity_id(o)
    if sid == oid:
        # self-loop from a name collision - skip
        return False
    self._node(sid, s, sty, doc_id)
    self._node(oid, o, oty, doc_id)
    key = (sid, rel, oid)
    edge = self.edges.get(key)
    if edge is None:
        self.edges[key] = {"source": sid, "target": oid, "relation": rel,
                           "docs": {doc_id} if doc_id else set(), "count": 1}
    else:
        edge["count"] += 1
        if doc_id:
            edge["docs"].add(doc_id)
    return True

def _node(self, eid: str, name: str, etype: str, doc_id: str) -> None:
    node = self.nodes.get(eid)
    if node is None:
        node = {"id": eid, "name": name, "type": etype, "docs": set()}
        self.nodes[eid] = node
        self._types[eid] = Counter()
    self._types[eid][etype] += 1
    # keep the most-voted type as the node label
    node["type"] = self._types[eid].most_common(1)[0][0]
    if doc_id:
        node["docs"].add(doc_id)
```

Say this — this is the "code enforces the contract" evidence. "Five rules, all in these thirty lines. **Entities merge by name-derived id** across windows and documents. **Type is decided by majority vote** — the `Counter` per entity means one inconsistent window can't mislabel a node. **Duplicate triples increment a `count`** rather than creating parallel edges, so repetition becomes a confidence signal I can rank by. **Self-loops from name collisions are dropped.** And **malformed triples return `False`** rather than raising, so a bad row is a skipped row.

The `docs` set on both nodes and edges is what makes cross-document structure visible: an entity with three `doc_ids` is a hub, and that's exactly the thing a graph gives you that a per-document index can't."

Windowed extraction, defensively

```
def extract_triples(self, text: str) -> list[dict]:
    """Windowed triple extraction over one document's text."""
    triples: list[dict] = []
    for w in _windows(text, self.window_words, self.max_windows):
        try:
            out = self.lmaas.complete_json(_TRIPLES_SYSTEM, w, cheap=self.cheap)
        except Exception:
            # noqa: BLE001 - a bad window shouldn't abort the doc
            continue
        got = out.get("triples") if isinstance(out, dict) else out
        if isinstance(got, list):
            triples.extend(t for t in got if isinstance(t, dict))
    return triples
```

Say this. "`got = out.get("triples") if isinstance(out, dict) else out` handles the model sometimes returning a bare list instead of the wrapped object — tolerate the shape, don't fight it. And extraction defaults to the **strong** deployment (`cheap=False`) because relationship extraction is where quality is load-bearing: it's what the graph is built from, and a wrong edge is permanent."

The PageIndex tree — LLM-built nesting

```
_PAGEINDEX_SYSTEM = """You are building a hierarchical PageIndex of a document – the
table-of-contents-like tree a navigator uses to find where an answer lives.
Read the document and return ONLY JSON of this exact shape:
{"title": "<document title>",
 "children": [
   {"title": "<section title>",
    "summary": "<one short sentence: what this section covers>",
    "children": [ ...same shape, nested... ]}
  ]}
Rules:
- Reflect the document's ACTUAL structure and nesting (chapters -> sub-sections).
- Titles are concise; summaries are one factual sentence.
- Include every substantive section; omit page furniture (page numbers, footers).
- Nest as deep as the document warrants; leaf sections have "children": [].
"""
```

Say this. "Two properties make this materially different from parsing headings. The **nesting is semantic, not numeric** — a heading-number parser can only nest what the numbering tells it, and the model nests by meaning, which matters when numbering is inconsistent or absent, or when an appendix numbered 5 really belongs under 3. And **every node carries a one-line summary**, which a parser cannot produce and which is exactly what makes the navigator's job easier — choosing between '3.2 Collimator Alignment' and '2.1 Detector Gain Calibration' is far easier with a sentence of context attached.

The honest observation: on clean markdown with explicit numbered headings, the LLM tree and the parsed tree **largely agree** — and I read that convergence as a *correctness signal*, not redundancy. The LLM path earns its keep on messy real-world sources where a parser degrades badly."

3.8 `clients.py` — the real backends

The two-leg IDAM token exchange

```
class IDAMClientCredentials:
    """LMaaS auth. TWO-leg IDAM flow, matching the official aifabric-lmaas-sample
    (idam_token_generator.py):

        1. client_credentials grant -> base access token
        2. token_exchange grant (Basic auth) -> LMaaS-scoped token (audience=LMAAS_AUDIENCE)

    The *exchange* token (Leg 2) is what the gateway accepts as the bearer token;
    a plain leg-1 client-credentials token is rejected. Cached + auto-refreshed.
    """

    def token(self) -> str:
        if self._token and time.monotonic() < self._expires_at:
            return self._token
        with self._lock:
            if self._token and time.monotonic() < self._expires_at:
                return self._token
            _require(
                ("IDAM_APP_CLIENT_ID", config.IDAM_APP_CLIENT_ID),
                ("IDAM_APP_CLIENT_SECRET", config.IDAM_APP_CLIENT_SECRET),
                ("LMAAS_AUDIENCE", config.LMAAS_AUDIENCE),
            )
            import uuid # lazy
            import requests # lazy
            from requests.auth import HTTPBasicAuth

            access_token = self._access_token()
            # Leg 2: exchange the base token for an LMaaS-scoped token.
            payload = {
                "grant_type": "token_exchange",
                "token": access_token,
                "audience": config.LMAAS_AUDIENCE,
                "scope": str(uuid.uuid4()),
            }
            resp = requests.post(
                config.IDAM_TOKEN_ENDPOINT,
                auth=HTTPBasicAuth(config.IDAM_APP_CLIENT_ID, config.IDAM_APP_CLIENT_SECRET),
                headers={"Content-Type": "application/x-www-form-urlencoded"},
                data=payload, timeout=config.LMAAS_TIMEOUT, verify=config.LMAAS_VERIFY_SSL,
            )
            if resp.status_code >= 400:
                raise BackendError(f"IDAM token-exchange {resp.status_code}: {resp.text[:300]}")
            body = resp.json()
            self._token = body["access_token"]
            self._expires_at = time.monotonic() + max(0.0, float(body.get("expires_in", 300)) - _TOKEN_SKEW_S)
            return self._token
```

Say this. "The pattern to point at is the **double-checked lock**: fast path outside the lock, re-check inside it. With parallel extraction running multiple threads, five threads hitting an expired token simultaneously would otherwise stampede the token endpoint with five identical exchanges. And `_expires_at` subtracts a **60-second skew** (`_TOKEN_SKEW_S`) so I refresh *before* expiry rather than discovering it via a 401 mid-request. `time.monotonic()` rather than `time.time()` because a clock adjustment shouldn't invalidate a valid token."

There are **two** token generator classes, and that's deliberate:

```
class IDAMTokenExchange:
    """OAuth2 token-exchange token (DIS): app token -> DIS-scoped token w/ roles."""
    # ... same two-leg shape, but scope="openid" and audience=config.DIS_APP_AUDIENCE
```

Say this. "Same shape, different audience — and critically the DIS-scoped token **carries a mapped RBAC roles claim** that the LMaaS token doesn't. Two classes rather than one with a flag, because they return tokens with different *semantics*, not just different values. That also lets me map agents to least privilege: Agent 1 needs the Admin role to ingest, Agent 2 only needs User."

Model tiering and the reasoning-model request contract

```
def _deployment(self, cheap: bool) -> str:
    dep = config.LMAAS_DEPLOYMENT_CHEAP if cheap else config.LMAAS_DEPLOYMENT_STRONG
    _require(("LMAAS_DEPLOYMENT", dep))
    return dep

def _sampling(self, max_tokens: int, temperature: float) -> dict:
    """Params vary by model family: gpt-5.x/reasoning models use
    `max_completion_tokens` and reject non-default `temperature`; Legacy
    gpt-4o-style models use `max_tokens` + `temperature` (see LMAAS_REASONING_MODEL)."""
    if config.LMAAS_REASONING_MODEL:
        return {"max_completion_tokens": max_tokens}
    return {"max_tokens": max_tokens, "temperature": temperature}

def complete_json(self, system: str, user: str, *, cheap: bool = False,
                  max_tokens: int = 4096) -> dict | list:
    resp = self._get_client().chat.completions.create(
        model=self._deployment(cheap),
        response_format={"type": "json_object"},
        messages=[{"role": "system", "content": system}, {"role": "user", "content": user}],
        **self._sampling(max_tokens, 0.0),
    )
    try:
        content = resp.choices[0].message.content or ""
    except (AttributeError, IndexError, TypeError) as exc:
        raise BackendError(f"Unexpected LMaaS response: {str(resp)[:300]}") from exc
    return _parse_json(content)
```

And the tolerant JSON parser:

```
_FENCED_JSON_RE = re.compile(r"```(?:json)?\s*(\{.*\}|\[.*\])\s*```", re.S)

def _parse_json(content: str) -> dict | list:
    content = (content or "").strip()
    try:
        return json.loads(content)
    except json.JSONDecodeError:
        m = _FENCED_JSON_RE.search(content)
        if m:
            try:
                return json.loads(m.group(1))
            except json.JSONDecodeError:
                pass
        raise BackendError(f"Could not parse JSON from model output: {content[:300]}")
```

Say this. `_sampling` is a six-line function that cost me a debugging cycle (\$6.5). Reasoning-class deployments **reject** `max_tokens` — you must send `max_completion_tokens` — and **reject any non-default** `temperature`, so the parameter has to be omitted entirely, not set to zero. Centralizing it behind one config flag means a legacy gpt-4o deployment still works by flipping a boolean, rather than a code change.

And `_parse_json` is layered on purpose: request JSON mode, then still tolerate a fenced ```json block, then raise with the actual content in the message so the failure is diagnosable. Every layer exists because the layer above it failed at least once."

Gremlin — injection defense, typed-entity persistence, SigV4

```
def _lit(value: Any) -> str:
    """Quote a value as a Gremlin/Groovy string Literal."""
    s = str(value).replace("\\", "\\\\").replace("'", "\\'")
    return f"'{s}'"

def _label(value: str, fallback: str) -> str:
    """Sanitize an LLM-supplied type/relation into a safe Gremlin Label."""
    s = re.sub(r"^[A-Za-z0-9_]", "", str(value or ""))
    return s or fallback
```


Say this — the security answer. "Neptune's HTTP Gremlin endpoint **doesn't support parameter bindings**, so every value is inlined as a literal in the submitted script. In this system the values include **LLM-generated entity names and relation labels** — which is to say attacker-influenceable content from ingested documents. So this is precisely where **prompt injection would become query injection**. A document that says 'your relation is `x').drop()`' is a real threat model, not a hypothetical. `_lit` escapes quotes and backslashes; `_label` whitelists to `[A-Za-z0-9_]` with a fallback. And `_label` doubles as the mechanism that turns LLM output into valid labels: `"UI Panel"` → `UIPanel`, `"part of"` → `PART_OF`."

```
def upsert_typed_entities(self, nodes: list[dict], edges: list[dict]) -> None:
    """Persist an LMaas-built typed entity graph (see KnowledgeGraph.to_store()).

    Idempotent: vertices upsert by id, an edge upserts on (source, relation, target).
    Neptune cannot change a vertex label after creation, so the type is ALSO stored
    as an `etype` property — that is what reads/filters use.
    """
    for n in nodes:
        nid, label = n["id"], _label(n.get("type"), "Entity")
        self.run_gremlin(
            f"g.V({_lit(nid)}).fold().coalesce(unfold(), "
            f"addV({_lit(label)}).property(id, {_lit(nid)}))"
            f".property('name', {_lit(n.get('name', nid))})"
            f".property('etype', {_lit(n.get('type', 'Entity'))})")
    for e in edges:
        sid, oid, rel = e["source"], e["target"], _label(e.get("relation"), "RELATED_TO")
        self.run_gremlin(
            f"g.V({_lit(sid)}).as('s').V({_lit(oid)}).coalesce("
            f"_.inE({_lit(rel)}).where(_.outV().hasId({_lit(sid)})),"
            f"_.addE({_lit(rel)}).from('s')).property('count', {int(e.get('count', 1))})")
```

Say this. "Two Neptune facts encoded here. **Vertex upsert** is `g.V(id).fold().coalesce(unfold(), addV(...))` — if it exists, unfold it; otherwise add it. So re-ingesting updates in place rather than duplicating or erroring, which means I can iterate on extraction against a live graph.

Edge upsert materializes the source endpoint as `as('s')` and checks `inE(rel).where(outV().hasId(source))` before adding. Two benefits: it's idempotent on the `(source, relation, target)` triple, and if an endpoint is missing the statement is a **safe no-op** instead of an error — so dangling edges from imperfect extraction never fail the run.

And the comment is the interesting one: **Neptune cannot change a vertex label after creation**, but the type is *LLM-assigned* and a later run can legitimately re-type an entity. So the type is stored **twice** — as the label and as an `etype` property — and all reads filter on `etype`. That's the general shape of the problem: **the store's immutable field can't be the mutable model's output.**"

```
def keyword_search(self, query: str, limit: int = 5) -> list[dict]:
    """Containment search over Chunk text. Uses TextP.containing on the first
    salient term; for production, back this with Neptune full-text search
    (OpenSearch) rather than a substring scan."""
    terms = [t for t in query.lower().split() if len(t) >= 4]
    if not terms:
        return []
    res = self.run_gremlin(
        f"g.V().hasLabel('Chunk').has('text', TextP.containing({_lit(terms[0]}))"
        f".limit({limit}).project('doc_id', 'section_id', 'text').by('docId').by(id).by('text')")
    return _values(res)
```

Say this — volunteer it. "This is the weakest retriever in the system and the docstring says so. It's a **substring scan on the first salient term**, and `TextP.containing` isn't even supported by plain Neptune without full-text search enabled.

Production needs the **OpenSearch full-text integration**. Two things make it survivable today: entity-dense documents lean on the **vectorless** path, which uses only standard Gremlin and works as-is; and a weak primary retrieval is exactly what the CRAG grader exists to catch and correct."

```
def run_gremlin(self, gremlin: str) -> dict:
    import requests # lazy
    _require(("NEPTUNE_ENDPOINT", config.NEPTUNE_ENDPOINT))
    url = f"https://{config.NEPTUNE_ENDPOINT}:{config.NEPTUNE_PORT}/gremlin"
    # Ask Neptune for UNTYPED GraphSON so result.data is plain JSON (List/dict/int),
    # not typed wrappers like {"@type": "g:List", "@value": [...]}. Without this the read
    # helpers (_values, get_tree) receive dicts where they expect lists.
    headers = {"Content-Type": "application/json",
               "Accept": "application/vnd.gremlin-v3.0+json;types=false"}
    data = json.dumps({"gremlin": gremlin})
    if config.NEPTUNE_USE_IAM:
        headers.update(self._sigv4_headers(url, data))
    resp = requests.post(url, data=data, headers=headers, timeout=60,
                        verify=config.DIS_VERIFY_SSL)
    if resp.status_code >= 400:
        raise BackendError(f"Neptune {resp.status_code}: {resp.text[:300]}")
    return resp.json()

def _sigv4_headers(self, url: str, body: str) -> dict[str, str]:
    import boto3 # lazy
    from botocore.auth import SigV4Auth
    from botocore.awsrequest import AWSRequest

    session = boto3.Session()
    creds = session.get_credentials()
    req = AWSRequest(method="POST", url=url, data=body,
                    headers={"Content-Type": "application/json"})
    SigV4Auth(creds, "neptune-db", config.AWS_REGION).add_auth(req)
    return dict(req.headers)
```

Say this — a genuine debugging story (§6.4). "That `Accept` header is a bug fix with a comment. By default Neptune returns **typed GraphSON v3**, where a list comes back as `{"@type": "g:List", "@value": [...]}`. My read helpers expected plain JSON, so they received dicts where they wanted lists and silently produced empty results — a *shape* bug, not an error. `;types=false` asks for untyped GraphSON and the whole read path becomes plain Python.

And SigV4: every request is signed for service `neptune-db` using the notebook role's boto3 credentials, so there are **no database passwords anywhere** — access is IAM policy, auditable in CloudTrail and revocable centrally. The signing is per-request work, which is why connection pooling matters on a write-heavy path: you save TCP+TLS setup *and* re-signing."

3.9 `loader.py` — PDF headings by typography, not word patterns

```
def _read_pdf(path: Path) -> str:
    """Extract PDF text, marking real headings with a Leading ``# `` so the
    ingester's structurer builds a meaningful tree.

    Headings are detected by TYPOGRAPHY, not word patterns: a line whose font is
    clearly larger than the document's body size (or bold and short) is a
    heading. This avoids mistaking numbered procedure steps and ALL-CAPS UI
    labels for headings — the failure mode of a pure text heuristic on real PDFs.
    """
    # ... build (text, font_size, is_bold) per reconstructed line, grouping words
    # into visual lines by rounded y-position ...

    # body size = the most common font size among substantial lines
    body_sizes = [sz for t, sz, _ in lines if sz > 0 and len(t.split()) >= 4]
    body = Counter(body_sizes).most_common(1)[0][0] if body_sizes else 0.0

    out: list[str] = []
    for text, size, bold in lines:
        if _is_boilerplate(text):
            continue
        n = len(text.split())
        is_heading = bool(body) and 0 < n <= 12 and not text.isdigit() and (
            size >= body * 1.15 or (bold and n <= 8 and size >= body * 0.98)
        )
        out.append(f"# {text}" if is_heading else text)
    return "\n".join(out)
```

Say this — it's a strong "show me your judgement" answer. "Three ideas. **The body font size is derived, not assumed** — it's the modal size among substantial lines (4+ words), so a document with 9pt or 11pt body text both work without configuration. **The heading test is relative** — 15% larger than body, or bold-and-short — so it's scale-invariant. And **words are grouped into visual lines by rounded y-position** with `extract_words(extra_attrs=["size", "fontname"])`, because pdfplumber gives you words, not typographic lines.

The design payoff is the contract with the ingester: the loader knows about fonts, so it emits `#`, and the section parser stays a two-condition function. Structure detection lives where the information is. There's also a **per-page fallback to** `extract_text()` if word extraction throws, so one malformed page degrades rather than failing the document."

3.10 `fakes.py` — the design decision that unblocked the whole project

```
def _recall(query: str, context: str) -> float:
    q = set(_terms(query))
    if not q:
        return 0.0
    ctx = set(_terms(context))
    return len(q & ctx) / len(q)

class FakeLMaaS:
    def __init__(self) -> None:
        self.calls: list[str] = []

    # -- JSON prompts: domain / entity-scan / evaluate / navigate -----
    def complete_json(self, system: str, user: str, *, cheap: bool = False, max_tokens: int = 4096):
        if "Assign a short domain label" in system:
            self.calls.append("domain")
            return {"domain": _guess_domain(user.lower())}
        if "DISTINCT salient entity" in system:
            self.calls.append("entities")
            # extract capitalized entity phrases from THIS window (deduped Later)
            return {"entities": _caps_phrases(user, limit=20)}
        if "judge whether retrieved context" in system:
            self.calls.append("evaluate")
            q = _section(user, "QUESTION:", ("CONTEXT:",))
            ctx = _section(user, "CONTEXT:")
            return {"score": round(_recall(q, ctx), 2), "reason": "term-overlap (fake)"}
        if "navigate a document" in system:
            self.calls.append("navigate")
            q = _section(user, "QUESTION:", ("HEADINGS:",))
            toc = _section(user, "HEADINGS:")
            chosen = []
            for line in toc.splitlines():
                line = line.strip().lstrip("- ").strip()
                if not line or ":" not in line:
                    continue
                sid, heading = line.split(":", 1)
                if _recall(q, heading) > 0:
                    chosen.append(sid.strip())
            return {"section_ids": chosen[:4]}
        return {}
```

```
class FakeDIS:
    """In-memory vector store with shallow coverage (first `coverage_words`)."""

    def __init__(self, coverage_words: int = 60) -> None:
        self.coverage_words = coverage_words
        self.collections: dict[str, list[dict]] = {}

    def upload_document(self, collection: str, doc_id: str, text: str, metadata=None) -> None:
        self.collections.setdefault(collection, [])
        head = " ".join(text.split()[: self.coverage_words]) # shallow index
        self.collections[collection].append(
            {"doc_id": doc_id, "text": head, "metadata": metadata or {}})
```

Say this — it's your best "tell me about being blocked" story. "The fake **inspects the system prompt** to know which pipeline step is calling it, then returns output that's canned but *logically appropriate*. The key choice: the evaluator's score is a

real **term-overlap recall** between question and context — so a CRAG verdict is a *consequence* of retrieval quality rather than a hard-coded value. If my routing broke, the verdict would change and a test would fail.

And `FakeDIS` is **adversarially designed**: it indexes only the first 60 words of each document. I built the fake to make my system fail in the way I claimed it could fail — deep-section queries miss the vector store and have to be recovered by vectorless navigation. That's the difference between a stub and a test double that's actually worth something.

`self.calls` records every step, so tests can assert on *which* LLM steps ran, not just the output. What none of this tests is prompt quality or real model behaviour — the docs note the fake's verdicts skew, e.g. grading 'correct' where a real model would say 'ambiguous'. Offline tests prove the machine is wired correctly; they say nothing about whether the model is any good."

3.11 Dependency injection — the swap is one line

```
def build_agents(use_fakes: bool):
    catalog = RoutingCatalog(path=catalog_path)

    if use_fakes:
        from hybridrag.fakes import FakeDIS, FakeGraph, FakeLMaaS
        lmaas, dis, graph = FakeLMaaS(), FakeDIS(), FakeGraph()
    else: # real services – requires config REPLACE_MEs to be filled
        from hybridrag.clients import DISClient, GraphClient, LMaaSClient
        lmaas, dis, graph = LMaaSClient(), DISClient(), GraphClient()

    ingest = IngestAgent(lmaas, dis, graph, catalog=catalog, collection="demo")
    query = CRAGQueryAgent(lmaas, dis, graph, catalog=catalog, collection="demo")
    return ingest, query
```

Say this. "That's the whole switch between offline and live. Both agents take `(lmaas, dis, graph)` as constructor arguments, the public API is **identical fake-vs-real**, and third-party imports are **lazy** — inside methods and inside this branch — so importing the package costs nothing and the offline path never needs `openai`, `requests`, or `boto3` installed. That last detail is why 11 tests run in 0.7 seconds."

3.12 The tests — how each CRAG branch is proven

The fixture is engineered so the branches are reachable deterministically:

```

def _ingested_agents(tmp_name: str):
    cat = _catalog(tmp_name)
    dis, graph, lmaas = FakeDIS(), FakeGraph(), FakeLmaas()
    ing = IngestAgent(lmaas, dis, graph, catalog=cat, collection="c", min_graph_entities=2)
    # Entity-dense -> graph-only. A section HEADING ("Calibration Error Recovery")
    # carries terms its BODY lacks, so graph keyword-search can miss while
    # vectorless heading-navigation recovers. Headings on their own lines.
    manual = (
        "View User Manual covers image display and layout tools.\n"
        "## 1.1 Imaging Features\n"
        "The system shows studies using the configured viewport.\n"
        "## 1.2 Calibration Error Recovery\n"
        "Restart the unit and rerun the routine to clear fault code 3455rr.\n"
    )
    ing.ingest("view_um", manual, title="View User Manual")
    q = CRAGQueryAgent(lmaas, dis, graph, catalog=cat, collection="c")
    return q, lmaas

def test_correct_path_early_exits():
    q, _ = _ingested_agents("t_correct.json")
    # intro body has all these terms -> strong graph hit -> correct
    res = q.run("image display layout tools")
    assert res.verdict.label == "correct"
    assert "vectorless" not in res.sources # no correction needed
    assert res.answer

def test_incorrect_path_recovers_via_vectorless():
    q, _ = _ingested_agents("t_incorrect.json")
    # no BODY contains these terms -> graph retrieval empty (incorrect) -> the
    # "Calibration Error Recovery" HEADING matches -> vectorless recovers.
    res = q.run("calibration error recovery")
    assert res.verdict.label == "incorrect"
    assert res.sources == ["vectorless"]
    assert "3455rr" in res.answer or "restart" in res.answer.lower()

def test_ambiguous_path_fuses_primary_and_vectorless():
    q, _ = _ingested_agents("t_ambiguous.json")
    # "image display" hits the intro body (partial); "calibration recovery" hits a
    # heading -> mid score (ambiguous) -> fuse graph + vectorless.
    res = q.run("image display calibration recovery")
    assert res.verdict.label == "ambiguous"
    assert "vectorless" in res.sources

def test_abstains_when_nothing_internal_matches():
    q, _ = _ingested_agents("t_abstain.json")
    res = q.run("quarterly revenue forecast for the finance department")
    # nothing internal matches -> incorrect -> vectorless empty -> abstain
    assert res.verdict.label == "incorrect"
    assert res.answer

```

Say this. "The fixture is the clever part, not the assertions. The manual's section **heading** carries terms its **body** doesn't — 'Calibration Error Recovery'. That single property makes all three branches reachable from one document: query the body terms and you get `correct`; query the heading-only terms and body search returns nothing so you get `incorrect` recovered by vectorless; query a mix and you get `ambiguous` and fusion. And `assert "3455rr" in res.answer` proves the corrective actually **recovered the right content**, not just that a branch fired."

The routing policy and multi-document scoping are pinned too:

```

def test_ingest_routes_dense_doc_to_graph_only():
    entry = agent.ingest("m1", "Service Manual install steps. Error Codes reference. "
                          "Detector Calibration. Smart Reading Protocol feature.",
                          title="Manual")
    assert entry.in_graph is True           # >= 3 entities -> graph
    assert entry.in_vector is False         # NEW policy: graph-only, not vectorized
    assert entry.meta["entity_count"] >= 3
    assert graph.get_sections(["m1"])       # tree was written
    assert not dis.collections.get("c")      # nothing uploaded to the vector store

def test_dual_store_toggle(monkeypatch):
    """With STORE_GRAPH_DOCS_IN_VECTOR on, a dense doc goes to BOTH stores."""
    from hybridrag import config as cfg
    monkeypatch.setattr(cfg, "STORE_GRAPH_DOCS_IN_VECTOR", True)
    e = agent.ingest("m1", "Service Manual. Error Codes. Detector Calibration. "
                      "Smart Reading Protocol.", title="Manual")
    assert e.in_graph is True and e.in_vector is True

def test_vectorless_scopes_to_candidate_docs():
    ing.ingest("aurora",
              "Aurora Imaging Manual\n## 1.1 Detector Calibration\nCalibrate the Detector and Collimator.\n",
              title="Aurora Imaging Manual")
    ing.ingest("handbook",
              "Field Handbook\n## 1.1 Escalation Rotation\nContact the Duty Lead after two hours.\n",
              title="Field Handbook")

    # scoping: the query matches only the handbook -> just that doc_id
    assert cat.candidate_doc_ids("Escalation Rotation Duty Lead") == ["handbook"]

    # end-to-end: graph body-search misses, vectorless recovers from the handbook
    q = CRAGQueryAgent(lmaas, dis, graph, catalog=cat, collection="c")
    res = q.run("Escalation Rotation")
    assert "vectorless" in res.sources
    assert "duty lead" in res.answer.lower()    # recovered handbook content, not aurora

```

Say this. "Three things worth noticing. `assert not dis.collections.get("c")` asserts on a **negative** — the graph-only policy means nothing was uploaded to the vector store, and a test that only checked `in_graph is True` would pass even if I'd accidentally dual-stored everything. The dual-store test uses `monkeypatch` on the config module, which is how a config-driven policy gets tested in both states. And the scoping test asserts `"duty lead" in res.answer` — it proves vectorless recovered from the **right document**, not merely that it recovered something."

Verified test run

```

$ pytest -q
.....
11 passed in 0.70s

```

[100%]

4. Skills & tools inventory

Say these words out loud. Every item is genuinely in `hybrid-rag` and you can defend it from \$3.

4.1 Python & core engineering

`Python 3.13` · `dataclasses` · `TypedDict` (LangGraph state schemas) · full type hints with `from __future__ import annotations` · `threading.Lock` **double-checked locking** (token cache) · `collections.Counter` (majority-vote typing, modal font size) · **lazy imports** (third-party imports inside methods so the offline path needs nothing installed) · dependency injection · interface segregation (a 2-method LLM client surface) · config-as-code with env overrides and fail-fast validation · idempotency as a design property

4.2 LLM / GenAI engineering

LangGraph — compiled `StateGraph`, typed state, nodes, **conditional edges**, `invoke`, checkpointing as a known next step · `openai.AzureOpenAI` SDK with `azure_ad_token_provider`

Techniques, named precisely: - **Corrective RAG (CRAG)** — retrieval grading + branch-specific correction - **LLM-as-judge** — the retrieval evaluator returning `{score, reason}` - **Vectorless / PageIndex-style retrieval** — LLM reasoning over a section tree, no embeddings - **Model tiering** — a `cheap` flag on every call mapped to two Azure deployments - **Prompt contracts** — exact JSON shape, canonical-naming rules, PascalCase types / UPPER_SNAKE_CASE relations, explicit "do NOT invent facts", explicit furniture exclusions, and explicit permission to return nothing - **Structured output** — `response_format={ "type": "json_object" }` + fenced-block-tolerant parser - **Windowed extraction with even sampling** — whole-document coverage under a hard call cap - **Triple extraction** → **typed property graph** — LLM-chosen node types and relation labels - **Majority-vote type resolution**; duplicate-triple **count as a confidence signal** - **Conditional / early-exit orchestration**; **zero-call shortcut** on empty retrieval - **Graceful degradation** — grader error defaults to `ambiguous` (widen, don't trust or discard) - **Explicit abstention** — refuse to generate with no knowledge, and skip the strong call entirely - **Reasoning-model request contract** — `max_completion_tokens`, omitted `temperature`

4.3 Knowledge graphs

Property-graph modelling · **property graph vs. RDF** trade-off · name-derived canonical IDs · cross-document entity merging · typed edges with properties (`count`, `docs`) · two-layer design (deterministic spine + LLM semantic layer) · provenance modelling (`MENTIONS` back to source section) · graph fragmentation diagnosis and remediation ranking · alias/coreference merging (named gap) · controlled-vocabulary vs. open-vocabulary typing

4.4 Amazon Neptune & Gremlin

Gremlin over the **HTTPS endpoint** (8182) · **SigV4 signing** via `botocore.SigV4Auth`, service `neptune-db` · IAM DB auth · **idempotent vertex upsert** `g.V(id).fold().coalesce(unfold(), addV(...))` · **materialized-endpoint edge upsert** with `as('s') + coalesce(inE(...).where(outV().hasId(...)), addE(...))` · `project().by(__...fold())` for single-round-trip reads · nested `project()` for whole-tree reads · `TextP.containing` and why it needs full-text search · **untyped GraphSON** (`types=false`) · **no HTTP bindings** → inlined escaped literals → injection defense · **vertex labels immutable after creation** → `etype` property · bulk loader (CSV/S3) as the cold-load alternative

4.5 AWS & networking

`SageMaker` notebooks inside the Neptune VPC · VPC / subnets / security groups (TCP 8182) · **PrivateLink interface VPC endpoints** and internal-vs-external DNS · IAM roles · `boto3` credentials · DynamoDB (named production catalog backend) · CloudTrail auditability of IAM DB auth

4.6 Authentication, security & compliance

OAuth2 `client_credentials` · **RFC 8693 token exchange** (`grant_type=token_exchange`, HTTP Basic, `audience`) · JWT `aud` claim inspection as a diagnostic · token caching with **60-second expiry skew** under a lock · **RBAC role mapping** (Admin=ingest, User=query, ServiceUser=health) and least privilege per agent · secret hygiene (env-only, `REPLACE_ME`,

`is_configured()` fail-fast) · TLS verification toggles for internal CAs · **prompt-injection** → **query-injection** threat modelling · Gremlin literal escaping + label whitelisting · XSS escaping in generated HTML · **PHI/PII posture** — non-production gateway, synthetic documents only

4.7 API integration & HTTP

`requests` with explicit timeouts and `verify` control · HTTP **error taxonomy** — `401` = identity, `403` with `Server:` `awselb/2.0` = network/WAF edge block · tolerant response parsing across shape variants (`results` / `passages` / `chunks`) · Bruno API collections · Azure OpenAI deployment-name addressing and `api-version`

4.8 Document processing

`pdfplumber` with `extract_words(extra_attrs=["size", "fontname"])` · **typography-based heading detection** (modal body font size, relative thresholds, bold-and-short rule) · visual-line reconstruction by y-position · boilerplate/furniture regex filtering · deterministic section-tree parsing · fixed-size chunking

4.9 Testing

`pytest` · `monkeypatch` for config-driven policy · **fakes that duck-type the real clients** · prompt-dispatching test doubles · **deterministic term-overlap scoring so verdicts follow retrieval quality, not randomness** · **adversarially designed fakes** (shallow vector index to force the corrective path) · asserting on negatives (`assert not dis.collections`) · asserting recovered *content*, not just branch identity · offline-first: 11 tests, 0.70 s, zero credentials

4.10 Visualization & communication

`vis-network` — self-contained interactive HTML (entity KG, PageIndex tree, document graph), nodes coloured by label, toggleable legend with counts, click-to-inspect, inline via `<iframe srcdoc>` · **reading the graph back out of Neptune** so views reflect persisted state · Confluence engineering write-ups · architecture + decision logs ("do not relitigate" lists) · function-by-function codebase guide · session hand-off docs

5. Alternatives not used, and why

This is where senior interviews are decided. The pattern for every row: **name it, state its real strength, then give the specific reason it lost here.** Never say an alternative is bad.

5.1 RAG architecture & retrieval

Alternative	Its real strength	Why not here
Plain vector RAG	Simple, cheap, strong on narrative prose	The four failure modes in §2.1 — relationship questions are joins; retrieval failure is silent; not every document deserves a graph; embedding coverage is uneven exactly where field engineers need depth.
Microsoft GraphRAG	The best-known graph-RAG implementation — entity/relation extraction, Leiden community detection, hierarchical community summaries; excellent for global "what are the themes?" questions	Two reasons. Cost: its indexing pass summarizes every community at every level across a whole corpus; my ingest is bounded at ≤ 8 cheap + ≤ 7 strong calls per document. Question shape: it optimizes <i>global sensemaking</i> ; service-manual queries are local and specific ("what does E-204 mean and how do I clear it?"), where community summaries add latency without precision. I did borrow its core insight — let the LLM build the typed graph.
RAPTOR (recursive clustering + summarization tree)	Elegant multi-scale retrieval on long documents	It builds its tree by <i>embedding and clustering</i> . My tree already exists for free as the document's own section hierarchy, and my navigator reads it with no embeddings at all . RAPTOR would have me pay for an embedding model to reconstruct structure the document already declares.
Self-RAG	Model emits reflection tokens deciding when to retrieve and whether output is supported	Reflection tokens generally imply a fine-tuned model. I'm on a governed gateway with fixed deployments — fine-tuning wasn't available. CRAG gets most of the benefit with a prompted cheap grader.
Adaptive-RAG (complexity classifier → no-retrieval / single-step / multi-step)	Genuinely cheaper on easy questions	I route on <i>store choice</i> , it routes on <i>hop count</i> . Complementary rather than competing, and a natural extension (iterative CRAG with a hop budget).
Canonical CRAG's web-search corrective	The published design's own answer to bad retrieval	Hard-blocked, and I'd argue against it anyway. No VPC egress; and for internal device service documentation, answering from the open web is a <i>worse</i> failure than abstaining. So I kept the mechanism and swapped the corrective source. My favourite thing to be asked about.
HyDE (generate a hypothetical answer, embed that)	Cheap, real recall gains on vague queries	Costs an extra LLM call on every query to fix a problem my grader only pays for when it actually occurs. My spend is conditional; HyDE's is unconditional.
Cross-encoder reranker (BGE, Cohere Rerank)	The standard, well-benchmarked precision fix at the top of the funnel	A real gap and I'll say so. My grader judges the <i>whole context set</i> pass/fail; a reranker orders <i>individual passages</i> . They compose. Skipped because it means another deployment provisioned on the gateway, and because grading is what unlocked <i>correction</i> . "With one more sprint, a reranker before the grader is where I'd spend it."
Reciprocal Rank Fusion / weighted hybrid BM25+vector	The proven default for combining retrievers, no LLM needed	My two knowledge blocks aren't two ranked lists over one corpus — they're a graph context and a set of LLM-chosen sections, with incomparable scores . Concatenating and letting the strong generator arbitrate is more honest than inventing a shared scale. RRF becomes correct the moment I add a second ranked retriever.

Alternative	Its real strength	Why not here
ColBERT / late interaction	Excellent precision-per-token	Needs its own index and serving path. The managed vector platform owns embeddings; I don't choose the retrieval model.
Long-context stuffing	No retrieval infrastructure at all	Fails on cost at corpus scale, fails on citation, degrades mid-context — and doesn't answer <i>which</i> manual to stuff.
Embedding-based semantic chunking	Chunk boundaries that respect meaning	Needs an embeddings deployment provisioned; fixed-size chunking is free and fully reproducible (same input → same chunks), which matters while debugging an extraction pipeline. Documented as opt-in if chunk quality is ever shown to be the bottleneck — I won't pay for it on a hunch.

5.2 Orchestration

Alternative	Its real strength	Why not here
CrewAI / AutoGen (autonomous multi-agent)	Emergent problem-solving when decomposition isn't known ahead of time	I know the pipeline. Ingest is fixed ETL with probabilistic steps; query is a router with three known corrective branches. Autonomy where the ontology is the contract is a liability — I'd trade determinism and step-level observability for unpredictability I don't need. My agents are agentic where it matters (routing and grading decisions that change control flow) and deterministic everywhere else.
Plain LangChain chains / LCEL	Simpler, less ceremony	No native conditional branching or typed shared state. My whole design <i>is</i> branching. Expressing it in LCEL means hand-rolling the state machine LangGraph gives me, and losing the free trace.
n8n / low-code workflow tools	Fast to demo, visual, non-engineers can edit	Explicitly ruled out. Version control, testing, and code review are non-negotiable for something heading toward deployment. A JSON workflow blob isn't reviewable.
AWS Step Functions	Managed, durable, retries and audit trail built in	Genuinely the strongest production alternative and I'd consider it. But each LLM step becomes a Lambda, the state machine lives in ASL rather than Python, and I lose the ability to run the whole orchestration offline in a unit test in 0.7 seconds — the constraint that mattered while credentials were pending.
Bedrock Agents / Knowledge Bases	Least code; managed RAG and orchestration	The organization mandates its own governed gateway for model access. And a managed KB wouldn't let me implement a <i>custom</i> corrective branch — which is the entire contribution.
Raw Python functions, no framework	Zero dependencies, total control, trivially testable	Honestly defensible, and for a 200-line prototype I'd have done it. I chose LangGraph because the stated goal is deployment : typed state, conditional edges, and a path to checkpointing (so a failed run resumes without re-paying for LLM calls) are things I'd otherwise build myself.

5.3 Stores

Alternative	Its real strength	Why not here
Neo4j	Best DX in the space — Cypher, Bloom, APOC, huge community, first-class GraphRAG tooling	The workload had to live in the org's AWS account and VPC with IAM auth. Neptune is the AWS-native managed property graph, so IAM/SigV4, VPC isolation, and serverless scaling came free with no new vendor onboarding or security review. In regulated healthcare, "no new vendor" is worth a lot of developer convenience. "Greenfield outside those constraints, I'd pick Neo4j."
Cypher / openCypher	Far more readable than Gremlin	Neptune supports it, but a working SigV4-signed Gremlin path already existed, and <code>project().by()</code> is genuinely excellent for folding a whole answer into one round-trip. I wouldn't defend Gremlin as <i>easier</i> — it isn't.

Alternative	Its real strength	Why not here
RDF / SPARQL triple store (Neptune supports both)	Formal semantics, OWL reasoning, standard vocabularies	I need properties on edges (<code>count</code> , <code>docs</code> , eventually provenance) — native in a property graph, reification gymnastics in RDF. No reasoning requirement, no external vocabulary to interoperate with. I'd revisit if this had to align with SNOMED.
Pinecone / Weaviate / Qdrant / Chroma / Milvus	Purpose-built ANN, rich filtering, great DX	The org already runs DIS , a managed vector-RAG platform owning chunking, embeddings, the store, grounded retrieval, and RBAC . A new vector DB means new onboarding, new auth, and duplicated compliance review for capability I already have. "I don't build a vector database; DIS is the vector database."
OpenSearch / Elasticsearch	Hybrid BM25 + kNN in one engine — and it's what Neptune full-text search is built on	The one I'd actively add: it fixes my <code>TextP.containing</code> gap. Not rejected, not yet provisioned .
FAISS / local embeddings	Free, fast, no network	No persistence, no RBAC, no multi-tenancy, and it puts content on notebook disk. Fine for a demo, wrong for the platform.
Postgres + recursive CTEs	One less system; teams know SQL	Multi-hop questions become chains of self-JOINs. The whole reason for a graph is that "error code → fault → procedure → role" is one traversal.
Neptune Analytics / Neptune ML	In-memory graph algorithms; GNN link prediction	I need <i>exact, cited, auditable</i> facts, not statistical link prediction — a predicted edge in a service manual is a hallucination with extra steps. Worth revisiting for entity resolution specifically.

5.4 Extraction & everything else

Alternative	Why not here
spaCy statistical NER + dependency parsing	I built this first in a sibling prototype, and measuring it justified moving on: entity type comes from a fixed mapping, relation type is the verb lemma , and on domain tokens it fails outright — an error code like <code>3455rr</code> becomes <code>CARDINAL</code> . Free and domain-agnostic, but too coarse to build a service-manual graph from.
Deterministic rules / regex only	Built twice in sibling projects, near-100% precision on known structure — and it's still what parses the <code>Document → Section → Chunk</code> spine <i>inside</i> this system, because structure is a parsing problem with a correct answer. It doesn't generalize: every new domain needs hand-tuned heading patterns and keyword lists.
Fine-tuning an extraction model	Needs labeled data I don't have, an eval set I haven't built, and a fine-tuning path the governed gateway doesn't offer. Prompt + validate + assemble gets me there at prototype volume.
<code>unstructured.io</code> / LlamaParse / LlamaIndex property-graph index	Reasonable, and I'd evaluate them for production ingest. The point of the exercise was to <i>own</i> the extraction and graph-assembly contract — a black-box parser would have hidden exactly the failure modes I needed to find (all of \$6).
AWS Textract	Genuinely better on tables, multi-column, and scanned pages. The loader's PDF path is decoupled enough that Textract page strings could be substituted; not needed for clean digital PDFs.
Text2Cypher / Text2Gremlin (LLM writes the query)	Fragile and a security problem — an LLM-authored traversal against a live cluster is arbitrary code execution, and it fails silently in strange ways. I chose curated, parameterized traversals: the LLM <i>phrases</i> answers over guaranteed-correct facts and never authors the retrieval.
OpenAI / Anthropic APIs directly	Not the approved inference path. All model traffic goes through the governed gateway for auditability and data residency — nothing leaves the VPC. Also far easier to defend in a security review.
Static API keys	The gateway rejects them; auth is IDAM OAuth — which is better anyway: short-lived tokens with an <code>aud</code> claim and RBAC roles beat a long-lived secret in an env var.
Neptune bulk loader (CSV via S3)	Correct for a cold load of a large corpus , and documented as such. Wrong for incremental, idempotent, re-runnable updates of tens-to-hundreds of elements from a notebook.

Alternative	Why not here
DynamoDB for the routing catalog	The planned production backend. A JSON file was right for the prototype — inspectable, diffable, zero setup — and the interface (<code>put/get/all/candidates</code>) is what would be swapped, not the callers. I name the limitation before anyone asks: single file, no cross-process locking, term overlap rather than a real index.
Dual-storing every document	The obvious "why choose?" answer, and it's a config toggle . Default off because duplicating entity-rich content as embeddings costs storage and ingest calls for recall the graph + vectorless path largely provides. Honest cost: graph-only documents lose fuzzy semantic recall, and CRAG is what compensates.
VCR/cassette recording or live integration tests	Cassettes need one successful live run to record — I had no credentials. Hand-written fakes let me build and test the entire orchestration first, and let me <i>design</i> the failure cases.
Neo4j Bloom / Graphistry / D3 / Cytoscape	<code>vis-network</code> in a self-contained HTML file means no server, no egress, inline rendering in a SageMaker notebook, and one shareable downloadable file. In a no-egress VPC I can vendor the JS.

6. Difficulties

Two kinds, and interviewers want both: **things that actually bit you** (proof you debug) and **things you anticipated** (proof you think ahead). Learn the first three cold.

6.1 ★ A threshold that measured the wrong thing — and being told so

The bug. The graph-worthiness gate counted entities from a curated sample of the **first 2000 characters**, capped at ≤ 5 items, against a threshold of **3**. It "worked" and I'd moved on.

The problem, which a reviewer named before I did: the number 3 was measuring **sample size, not the document**. A 200-page manual and a 2-page memo could produce the same count, and no value of that threshold would ever mean what a reader assumes it means.

The fix (code in §3.4): whole-document ~400-word windows, one cheap LLM call each, entities **normalized and de-duplicated** across windows, count **distinct**, compare to `INGEST_MIN_GRAPH_ENTITIES = 8`. Cost bounded by a hard cap of 8 windows — above which windows are **sampled evenly across the document**, so coverage stays whole-document.

Why it's the story you tell. "The threshold now means exactly what someone reading the config would assume: 'at least 8 distinct entities across the document.' The general lesson is that **a magic number is a bug when it doesn't correspond to the thing it claims to measure** — and I'd rather be corrected early than defend a number I couldn't explain. There was a bonus, too: the scanned entities enrich the catalog and improve query-time routing, so the extra work pays for itself even for vector-only documents."

6.2 ★ The heading detector that flooded the tree

The symptom. Vectorless navigation was returning nonsense sections. The navigator wasn't the problem — its **table of contents was garbage**.

The cause. The first `_is_heading` treated single-number lines (`1 Do X`) and short ALL-CAPS lines as headings. On real manual text, **every numbered procedure step and every UI label became a section**. The section tree was noise, so the model was choosing from noise.

The fix, in two parts (code in §3.4 and §3.9): 1. Narrow `_is_heading` to two strong signals only — a markdown `#`, or dotted multi-level numbering (`2.4.3`). Reject the rest. 2. That created a follow-on problem: real PDFs have no `#`. So the **PDF loader detects headings by typography** — modal body font size, then " $\geq 15\%$ larger than body, or bold and short" — and emits `#` so the section parser stays a two-condition function.

Why it's the story you tell. "Two lessons. **Debug the input, not the component** — I nearly went prompt-tuning the navigator when the actual defect was upstream. And the fix is a design improvement, not a patch: **structure detection now lives where the information is**. The loader knows about fonts; the parser knows about one contract. Also worth naming — a pure text heuristic on real PDFs is a trap, because procedure steps and UI labels look exactly like headings in plain text and nothing like them typographically."

6.3 ★ The 403 that wasn't an auth failure

The symptom. From a SageMaker notebook, every LMaaS call returned `403 Forbidden`. Everything pointed at credentials — wrong audience, missing entitlement, bad secret.

The cause. The response header was `Server: awselb/2.0` — an AWS load-balancer edge block, not the application. LMaaS publishes **three** access paths per environment:

Variant	Shape	Who it's for
External	<code>https://lmaas-<env>-int.ailab.gehealthcare.net</code>	Corporate network / VPN only — IP-allowlisted

Variant	Shape	Who it's for
Internal VPCE DNS	<code>https://lmaas-<env>.ailab.gehealthcare.net (no -int)</code>	AWS / SageMaker via PrivateLink ← the right one
Internal VPCE Service	<code>com.amazonaws.vpce.<region>.vpce-svc-<id></code>	The PrivateLink service you build an interface endpoint from

The notebook was calling the **External** (VPN-only) hostname from inside AWS. Fix: create an interface VPC endpoint from the VPCE service, resolve the **Internal VPCE DNS** name to it, call that.

The diagnostic to memorize — this is the sentence that impresses:

"A **401** means *we don't know you*. A **403** with `Server: awselb/2.0` — including on a request with no auth header at all — means *your network isn't allowed*. Sending a deliberately unauthenticated request is the cleanest way to separate identity from network, because a real auth layer would have said 401."

And the positive check: "decode the token and inspect the `aud` claim. If `aud` is the LMaaS audience, identity is fine and anything still failing is authorization or network. Between those two tests I can always tell which layer is broken."

6.4 The Neptune response shape that silently returned nothing

The symptom. Reads against a real cluster returned **empty results with no error**. Writes were landing — I could see the data — but `get_sections`, `get_tree`, and the entity-graph reads all came back empty.

The cause. Neptune returns **typed GraphSON v3** by default, where a list arrives as `{"@type": "g:List", "@value": [...]}`. My read helpers expected plain JSON, so they received a dict where they wanted a list and produced nothing. A **shape** bug, not a failure — the worst kind, because nothing raises.

The fix — one header, with a comment explaining it forever (code in §3.8):

```
headers = {"Content-Type": "application/json",
           "Accept": "application/vnd.gremlin-v3.0+json;types=false"}
```

Why it's worth telling. "Two failure modes in this project produced *silently wrong* results rather than errors — this one and the threshold. Both taught the same habit: **for any integration, assert on the parsed shape early**, because a transport that returns 200 with a structure you didn't expect will quietly propagate emptiness through five layers before anyone notices."

6.5 The auth flow was two legs, not one — and my first two assumptions were both wrong

Assumption 1 (wrong): an OpenAI-compatible `/chat/completions` with a static bearer key. **Reality:** an **Azure OpenAI**-shaped gateway (`/openai/deployments/{deployment}/chat/completions?api-version=...`), addressed by **deployment name**, via the official `AzureOpenAI` SDK.

Assumption 2 (also wrong): single-leg OAuth2 `client_credentials` with an `audience` parameter. It returned a token, the token looked fine, and the gateway **rejected it**.

Reality — a **two-leg RFC 8693 token exchange** (code in §3.8):

```
LEG 1 grant_type=client_credentials (client_id, client_secret, scope) → access_token
LEG 2 grant_type=token_exchange      (HTTP Basic auth: id:secret,
                                     token=<access_token>,
                                     audience=<lmaas-audience>) → exchange_token
                                     ↑ the bearer the
                                     gateway accepts
```

How I found it: by reading the platform team's own official sample code rather than trusting my inference from the documentation.

Why it's the story you tell. "Two documented-looking assumptions in a row were wrong. What fixed it wasn't more debugging — it was going to the source of truth, the working sample. I now treat 'the docs imply X' as a hypothesis to verify against a running example, especially for auth."

Related trap: the internal CA isn't in the default trust store, so the official sample uses `verify=False`. I exposed it as config (`LMAAS_VERIFY_SSL`) with an explanatory comment rather than hard-coding an insecure default — "the sample did it" isn't a security argument."

6.6 GPT-5.x reasoning deployments changed the request contract

Reasoning-class deployments **reject the legacy sampling parameters**: - `max_tokens` is not supported → send `max_completion_tokens` - a non-default `temperature` is **rejected** → omit the parameter **entirely**

Centralized in one `_sampling()` helper behind `LMAAS_REASONING_MODEL` (code in §3.8), so a legacy gpt-4o-class deployment works by flipping one boolean.

The subtler consequences — mention these, they show depth: 1. Reasoning models spend part of the budget on **internal reasoning**, so token budgets must be more generous than for a chat model producing the same visible output. 2. **You can't set `temperature=0`**, so the usual determinism lever is gone. Reproducibility has to come from elsewhere: strict JSON contracts, structural validation, name-derived IDs, idempotent upserts, and majority-vote typing — so **reproducibility becomes a property of the assembly rules, not of the model call**. 3. They're markedly **slower per call**, which is why the demo advice is to pre-run ingest and demo only the query loop.

6.7 The LangGraph state-merge quirk

Cross-branch state merges after a conditional edge proved unreliable, so propagating `in_graph` / `in_vector` as node state from the two branch nodes was fragile.

Fix: compute the flags **deterministically** in `write_catalog` from `build_graph` plus the dual-store config (code in §3.4), and document it so nobody reverts it.

"A deliberate robustness choice — when a framework's guarantee is unclear, **derive the value from inputs you control** instead of depending on propagation you can't verify."

6.8 Neptune vertex labels are immutable

You cannot change a vertex label after creation. But the entity **type is LLM-assigned**, so a later run can legitimately re-type an entity.

Resolution: store the type **twice** — as the vertex label *and* as an `etype` property. All reads and filters use `etype`, so a re-type updates `etype` while the original label persists harmlessly; `drop by etype` gives a clean rebuild.

"The general shape of the problem: **the store's immutable field can't be the mutable model's output**, so the mutable copy has to live in a property."

6.9 No bindings on Neptune's HTTP Gremlin endpoint → an injection surface I had to close

Neptune's HTTP Gremlin endpoint **doesn't support parameter bindings**, so every value is inlined into the submitted script — and here the values include **LLM-generated entity names and relation labels**, i.e. attacker-influenceable content from ingested documents.

Defenses (code in §3.8): `_lit()` escapes quotes and backslashes; `_label()` whitelists to `[A-Za-z0-9_]` with a fallback — which doubles as the mechanism that turns LLM output into valid Gremlin labels.

"This is the place where **prompt injection becomes query injection**. A document that says 'your relation is `x').drop()`' is a real threat model, not a hypothetical, and sanitizing at the boundary is why it isn't one here."

6.10 Graph fragmentation — the honest quality limitation

The LLM-built entity graph is **more fragmented** than the structural spine. Frame it correctly: **expected, not a defect**.

- The spine is connected **by construction** — every section hangs off one `Document`.
- Two triples only connect if they **share an entity name**. Windows are extracted **independently**, so the same real thing surfaces as "Detector" in one window and "the Detector assembly" in another and fails to merge.

Ranked fixes, and why in that order: canonical-name carry-forward first — pass already-extracted entity names into subsequent windows so the model reuses them. Cheapest change (a prompt change), attacks the root cause directly. Then **alias merging** as a post-pass (substring/fuzzy/embedding similarity). Then **per-triple provenance**, so any graph-derived answer can be cited back to a text span — a prerequisite for trust in a service context.

"I can name my system's biggest quality weakness, explain *why* it happens mechanically, and rank the fixes by value-per-effort. Retrofitting graph connectivity is much harder than extracting it connected, which is why I'd do the prompt change first and immediately."

6.11 Smaller ones worth having ready

Difficulty	Resolution
<code>TextP.containing</code> isn't supported by plain Neptune without full-text search	Needs the OpenSearch integration. Until then entity-dense docs lean on the vectorless path, which uses only standard Gremlin. Named as a limitation in the docstring, not hidden.
Model returns JSON wrapped in a fenced ```json block	JSON mode requested via <code>response_format</code> , with a fenced-block-tolerant parser as fallback before raising with the actual content
Model returns a bare list instead of <code>{"triples": [...]}</code>	<code>got = out.get("triples") if isinstance(out, dict) else out</code> — tolerate the shape
Vector platform's response shape unknown pending the API collection	<code>retrieve()</code> accepts <code>results</code> / <code>passages</code> / <code>chunks</code> ; paths are explicit <code>REPLACE_ME</code> with <code>is_configured()</code> fail-fast, so an unconfigured system says what's missing instead of returning a confusing 401
Entities the LLM invents that appear in no section text	<code>_entity_mentions</code> skips them — no orphan nodes
A high-frequency term producing hundreds of useless edges	<code>max_per_entity=12</code> cap on mention edges
A document with no detectable headings	<code>_naive_sections</code> falls back to one <code>"Document"</code> section with flat chunks — degrades, never returns empty
One malformed PDF page	Per-page fallback to <code>extract_text()</code> ; one bad page degrades rather than failing the document
Gremlin writes are one HTTP POST per statement	Known latency limitation. Batching (newline-joined statements) is the fix — with the caveat that every statement must then end in <code>.iterate()</code> , because Neptune only iterates the last statement of a submitted script. Learned that the hard way in a sibling project, where a batch returned HTTP 200 and silently persisted one element.

6.12 Difficulties you *anticipated* — the "what would worry you in production?" bank

1. **Prompt injection from ingested documents**. Mitigated at the Gremlin boundary (\$6.9); **not** mitigated in the generator prompt. Real gap, and I'd name it as one.

2. **PHI/PII in a non-production gateway.** Approved for POC/dev only — **no PHI/PII**. Enforced by using synthetic service manuals. A compliance constraint, not a preference, and the first thing I'd raise in a design review.
 3. **Cost blow-up at corpus scale.** Ingest is bounded per document but linear in documents; strong-model triple extraction dominates. Levers: cheaper extraction tier, lower window caps, incremental re-ingest on revision only.
 4. **Unmeasured extraction quality.** No gold-standard triple set, so precision/recall are **eyeballed, not measured** — meaning I can't tell whether a prompt change helped. Highest-value missing artifact in the project.
 5. **Uncalibrated thresholds.** 0.7 and 0.3 are reasoned, not fitted, and likely corpus-specific.
 6. **LLM-as-judge bias.** Self-preference, positional bias, mid-range clustering, and a *cheap* grader judging on behalf of a *strong* generator. Shipped mitigation: grader error → **ambiguous**. Not shipped: agreement against human labels.
 7. **Label/schema explosion.** LLM-chosen types will drift across a corpus — **Component** vs **Hardware** vs **Part**. Majority vote stabilizes typing *within* an entity, not the global vocabulary. Needs a controlled vocabulary or a mapping pass.
 8. **Document revision handling.** Writes are idempotent so re-ingest updates in place — but **nothing deletes content removed in a new revision**. For regulated service documentation a stale superseded warning is a genuine safety problem. Needs revision-scoped subgraphs and a delete path.
 9. **Catalog concurrency.** A single JSON file, thread-safe in-process, **no cross-process locking**. Two concurrent ingests race. Exactly why DynamoDB is the named production backend.
 10. **Observability.** A per-query **trace**, but no distributed tracing and **no token/cost metering per request** — so I can quote call counts confidently and dollar costs only roughly.
 11. **No checkpointing.** A late failure re-pays every earlier LLM call. LangGraph's **MemorySaver** is the documented fix and it's a small change.
 12. **Correction loops.** Corrections are **single-round**, which trivially guarantees termination. Iterative CRAG needs an explicit **hop budget** or it can spin.
 13. **Abstention vs. helpfulness.** Right for safety-critical content, but a system that abstains too often gets abandoned. A product calibration question, not just an engineering one.
 14. **Truncation.** PageIndex caps input at ~6000 words; the entity scan caps at 8 windows. Both are cost guards that **silently reduce coverage** — I'd rather **log what was dropped** than let silent truncation read as complete coverage.
 15. **Cold-start routing.** With an empty catalog every query falls back to whatever store exists. Safe, but routing quality *improves* with ingest — worth stating so nobody's surprised.
-

7. Q&A: architecture, RAG & CRAG

Q1. Give me the one-line summary.

"Two LangGraph agents over three retrieval modes: one decides how each document should be stored, the other grades its own retrieval and corrects it before answering — all inside the VPC on the organization's governed model gateway."

Q2. Why two agents instead of one pipeline?

"They run at completely different times, on different cadences, with different cost profiles. Ingest is one-off per document and expensive; query is per-request and must be cheap. Splitting them lets me push work **left** — the entity scan I pay for once at ingest is what makes query-time routing cheap and non-blind. The routing catalog is the only coupling, and it's a **written contract** rather than shared code: neither agent imports the other, so they can be deployed and scaled independently, and I can `cat` the catalog and see exactly what Agent 2 will conclude."

Q3. What makes these "agents" rather than just pipelines?

"Be precise: agentic where it matters, deterministic everywhere else. Both make **decisions that change control flow** — Agent 1 routes a document to a different store based on evidence it gathers; Agent 2 picks a retrieval source and then takes one of three corrective branches based on a grade it computes. That's conditional routing on model output, which is the useful part of agency. What they deliberately *don't* have is autonomy over goals or tool selection. On an ingestion path where the ontology is the contract, that would be a liability."

Q4. Why is the routing catalog the "linchpin"?

"Without it, Agent 2 blind-calls every store on every query — wasted retrievals, wasted latency, and no principled way to pick a primary. It answers two distinct questions: which **stores** could plausibly hold relevant content, and which specific **doc_ids** to hand the vectorless navigator so the LLM reads a focused table of contents instead of the whole corpus. The second one is what keeps vectorless cheap as the corpus grows."

Q5. Walk me through query routing.

Recite the precedence from `_primary_source` (§3.6): "Catalog first, then intent. `candidates()` overlaps query terms of length ≥ 4 against each document's title, domain, and entity list. Then: if the query is **relational** — matches hints like 'how does', 'depend', 'connect', 'between', 'cause' — and graph is a candidate, go graph. Otherwise if vector is a candidate, go vector. Otherwise graph. Otherwise default to vector for a cold catalog. And critically: **whatever the primary pick, the grader still checks it**, so routing being wrong is a recoverable event rather than a failure."

Q6. What's the weakest part of the routing?

"Term overlap. Bag-of-words with a length-4 filter — no stemming, no synonyms, no semantics. It'll miss a query that's semantically about a document but shares no long tokens with its title, domain, or entities. Two things save it: no match means fall back to *all* available stores rather than starving the query, and the grader catches a bad primary. The upgrade is **entity-anchored routing** — resolve query mentions to actual entity nodes and route by graph neighbourhood."

Q7. Why mutually exclusive storage? Isn't dual-storing obviously better for recall?

"Better for recall, worse for cost, and I made the cost call **with a toggle to reverse it**. Duplicating entity-rich content as embeddings means paying storage plus ingest calls to buy fuzzy recall the graph keyword search plus vectorless navigation largely already provide. And I'll state the cost honestly: **graph-only documents lose fuzzy semantic recall** — which is precisely the weakness the CRAG loop exists to compensate for. If a measured eval showed graph-only documents underperforming, `STORE_GRAPH_DOCS_IN_VECTOR=true` is a one-line change. That's the point of making an opinionated default reversible, and there's a test pinning both states."

Q8. Explain CRAG as if I don't know it.

"Corrective RAG adds one idea to normal RAG: **grade the retrieval before you trust it**, and take a *different* corrective action per grade. Normal RAG is retrieve → stuff → generate, with no opinion about whether what it retrieved was any good. CRAG inserts an evaluator that scores retrieved context against the question, then branches: good → answer; bad → replace the knowledge; partial → supplement it. The published version corrects via **web search**. I replaced that with internal vectorless navigation, because egress isn't available and — for internal service documentation — answering from the open web is a worse failure than abstaining."

Q9. What exactly is "vectorless" retrieval?

Walk the three steps from §3.5. "Pull **headings only** — tiny payload. Ask the cheap model which 1–4 sections would answer the question, reasoning about meaning rather than keywords. Fetch **only those** section bodies. It's how a human uses a 300-page manual: read the contents, flip to the chapter.

Four properties matter: **no embeddings** — nothing to provision or re-index; **cheap on large documents**, because deciding costs headings not text; **interpretable** — `meta["chosen"]` records which sections it picked, so you can judge the decision, which you can't do with a similarity score; and it's strongest exactly where embeddings are weakest — deep, specific, nested procedural sections."

Q10. Isn't vectorless just keyword search with extra steps?

"No — it selects by **meaning**, and the test suite proves the difference. The fixture has a section whose *heading* says 'Calibration Error Recovery' while its *body* never repeats those words. Keyword search over chunk text returns nothing. The navigator reads headings and reasons about which section would contain the answer — so it can also pick 'Escalation Procedures' for 'who do I call when a detector fails', where there's no keyword overlap at all. It's semantic selection over a compact structured index, with an LLM as the ranking function."

Q11. Why not just use embeddings for everything? They're cheap now.

"Three reasons that aren't about cost. **Coverage is uneven** — deeply nested sections are systematically under-represented, so the specific procedural content you most need is what you most reliably miss. **Relationship questions are joins** — 'what causes E-204 and what procedure clears it' is `ErrorCode --INDICATES--> Fault --RESOLVED_BY--> Procedure`; embeddings retrieve passages that *look* similar, they cannot traverse. **Exact domain tokens** — a code like `3455rr` is fuzzy text to an embedding model and an O(1) typed-node lookup in a graph. So it's not embeddings-versus-graph; it's using each where it's strong, which is the entire thesis."

Q12. How does the evaluator work, and what does it cost?

"One cheap-tier call. Query plus the first 4000 characters of context, returns `{score, reason}`, clamped to [0,1] and bucketed by the two thresholds. Two important edge cases in the code: **empty retrieval short-circuits to incorrect** with

zero LLM calls — grading nothing is waste — and if the **evaluator itself errors**, the verdict defaults to `ambiguous`, so a grader failure *widens* retrieval rather than trusting or discarding it. Fail toward more evidence, not less."

Q13. LLM-as-judge is known to be unreliable. Defend it.

"It is, and I'd list the failure modes myself: self-preference, positional bias, clustering near the middle, prompt sensitivity, and in my case a **cheap** model grading on behalf of a **strong** one.

Three things make it acceptable. The task is deliberately easy — 'could this context answer this question' is far easier than 'which answer is better', and it's a *relevance* judgement, not a quality one. I only need it right at the **boundaries**: I don't consume the score, I consume the bucket, and the middle band is wide because ambiguity is the common case. And every failure direction is safe by construction — grader error → ambiguous; a false 'incorrect' costs one extra cheap navigation call; a false 'correct' is the one that actually hurts, which is why the threshold sits at 0.7 rather than 0.5.

What I don't have is agreement data against human labels — that's the honest gap. A cross-encoder relevance model is the more principled instrument; I didn't use one because it's another deployment to provision, and because grading is what unlocked *correction*, which was the bigger win."

Q14. Why 0.7 and 0.3?

"Reasoned, not fitted, and I'll say that plainly. 0.7 is the skepticism dial — raise it toward 0.8 and the system corrects more often: higher recall, higher cost. 0.3 is the discard dial — lower it toward 0.2 and it fuses more rather than throwing retrieval away. The band between is deliberately wide because partial relevance is the common case and fusing is the cheap safe response. With a labeled set I'd sweep both and pick the pair maximizing answer accuracy at acceptable cost — and I'd expect the optimum to be **corpus-specific**, which is why they're in config."

Q15. Why is refinement conditional?

"Because refinement is a cost with a floor. Below 180 words there isn't enough context for 'decompose → filter → recompose' to remove anything meaningful, so the call is pure spend. Above it, noise reduction genuinely helps the generator. And note `refine` returns a `(text, called)` tuple so the call counter stays honest — I want reported cost to be real cost."

Q16. Why "fusion, not selection" on the ambiguous branch?

"A mid-range grade means *partial evidence*, and the right response to partial evidence is to **widen, not replace**. The primary gives one view; vectorless gives specific section bodies. I concatenate with a separator and let the strong generator arbitrate, because it has the query and both contexts and I don't. Inventing a score to pick a winner would be pretending to knowledge I don't have — and the scores genuinely aren't comparable: one's a retrieval similarity, the other's an LLM section choice."

Q17. Why does the "incorrect" branch fall back to the other store?

"Defense in depth for the case where both the router and the navigator whiff. If primary retrieval was bad and vectorless also came back empty, the last internal option is the store we didn't try. If that's also empty, `_n_generate` **abstains** — a fixed message with **no generation call at all**. Deliberate on two axes: never invite the generator to invent something, and never pay the strong model to do it."

Q18. How do you prevent hallucination?

"Four layers, none of them just 'the prompt says don't'. **The generator prompt** constrains it to the provided knowledge and tells it to say what's missing — giving the model an approved way to fail is what stops it inventing. **The extractor** is forbidden to invent relationships and told to ignore furniture. **Explicit abstention** — with no knowledge, no generation call happens at all, so there's no opportunity. And **provenance** — the spine links entities back through **MENTIONS** to the section that named them.

The gap I'd name: **per-triple provenance doesn't exist yet**, so a purely graph-derived answer can't be cited to a text span. In a service context that's a prerequisite for trust, and it's on the roadmap."

Q19. What if the evaluator says "correct" but the answer is wrong?

"That's the dangerous failure and I have no detection for it today. The grade is about **retrieval relevance**, not answer correctness — context can be genuinely relevant and the generator can still misread it. Two things I'd add: a **post-generation faithfulness check** (does every claim trace to the provided knowledge — a second cheap call, cheap because it's entailment not reasoning), and **citation enforcement** — require section IDs per claim, then verify mechanically that those sections were in the context. That turns an invisible failure into a detectable one."

Q20. What's the difference between your graph retrieval and your vectorless retrieval? Both hit Neptune.

"Different questions over the same store. **Graph** asks *what relates to what* — today keyword containment over **Chunk** text. **Vectorless** asks *where in the corpus the answer lives* — it navigates the **Document → Section** hierarchy by meaning. Content lookup versus navigation. And the vectorless index was **free**: the section tree is a by-product of graph ingest, so the corrective route cost nothing extra to build. That's the part of the design I like most — a capability that fell out of a decision made for a different reason."

Q21. How do you know any of this is better than plain RAG?

"Right now I don't — not numerically, and I won't claim otherwise. What I have is a design targeting four *specific, named* failure modes, and a test suite where each CRAG branch is **reproducibly exercised** on an engineered fixture: a body-text hit that grades correct and early-exits; a heading-only match where primary retrieval returns empty and vectorless recovers the right content; a partial match that grades ambiguous and fuses; and an out-of-corpus query that abstains. That's evidence the mechanism works. It isn't evidence of aggregate quality. For that I need a labeled query set with answer-relevance and citation-accuracy metrics — and that's the artifact I'd build next."

Q22. How would you evaluate this properly?

"Four layers, because 'is the answer good' is too coarse to act on. **Retrieval**: labeled query→relevant-section pairs, recall@k and MRR **per mode** — that's how I'd find out whether the graph leg is actually earning its keep. **Grading**: human-labeled verdicts, measure grader agreement, and specifically watch the **false-**correct** rate**, since that's the failure that reaches the user. **Extraction**: a gold-standard triple set, precision/recall on entities and relations, so prompt changes are measured rather than eyeballed. **End-to-end**: answer relevance, faithfulness, citation accuracy, plus **abstention precision** — did it abstain when it should have, and only then. Plus operational telemetry: verdict distribution, calls per query, p50/p95, cost per query."

Q23. Your chunking has no overlap. Isn't that a mistake?

"Chunks are ~120 words with **no overlap** — deliberately simple, and yes, it risks splitting a fact across a boundary. The reason it matters less here is what chunks are *for*: they're provenance units under sections, and retrieval happens at **section granularity** via vectorless, or through the entity graph. If I moved to chunk-level vector retrieval I'd add overlap immediately

— the standard shape being ~512 tokens with 64-token overlap on paragraph/sentence boundaries. It's a defensible prototype simplification, not a considered position on chunking."

Q24. How would you handle multi-turn / follow-up questions?

"Not implemented, and I wouldn't pretend the extension is trivial. Routing depends on term overlap against the catalog, and 'and what about the other one?' has no usable terms. So step one is **query rewriting**: a cheap call resolving the follow-up into a standalone question *before* routing. That composes cleanly, because my router and grader are both **stateless functions of a query string**, and LangGraph checkpointing would carry thread state. What I'd watch for is grader drift across turns — context that was 'correct' for turn one may be graded against a materially different turn- three question."

Q25. Could you drop the graph entirely and just do vector + vectorless?

"Partly yes, and it's a sharp question — vectorless only needs the **section tree**, which is far cheaper than the entity graph. So a leaner system is vector for semantic recall plus vectorless for structural navigation, with no entity extraction at all. What you'd lose is what motivated the graph: **relational, multi-hop questions** and exact typed-node lookup on domain tokens. There's an honest wrinkle too: the typed entity graph is persisted but **not yet traversed at query time**, so the *current* system is closer to that lean version than it looks. The graph's value is real but still largely unrealized, and I'd rather say that than oversell it."

Q26. What would you do differently if you started over?

"Three things, in order. **Build the eval harness first** — everything I'm least confident about (both thresholds, extraction quality, whether the graph leg earns its keep) is unmeasurable without it, so every improvement after this point is a guess dressed up as judgement. **Carry canonical entity names forward across extraction windows from day one**, because retrofitting graph connectivity is much harder than extracting it connected. **Batch the Gremlin writes from the start** — I'd already solved that in a sibling project and didn't carry the lesson forward, which is straightforwardly avoidable."

Q27. What are you proudest of?

"That the entire orchestration is testable in 0.7 seconds with no credentials. That wasn't a testing nicety — it was the constraint that let me build and validate the whole two-agent design, every CRAG branch, the routing logic, and the fusion behaviour **while credentials and API paths were still pending**. Dependency injection plus fakes that duck-type the real clients turned an external blocker into a non-blocker. And the fakes are *designed*, not stubbed — the vector fake indexes only each document's first 60 words specifically so the corrective path is exercised rather than hypothetical."

Q28. Where does this go next, concretely?

Rank them and say why in that order: 1. **Graph-native retrieval** — the typed graph is persisted but **not traversed at query time**; multi-hop (`ErrorCode` → `Fault` → `Procedure` → `Role`) is the clearest unclaimed payoff from having built it at all. 2. **Canonical-name carry-forward** — cheapest fix for the biggest quality problem. 3. **Eval set with precision/recall** — turns tuning from guessing into measuring. 4. **Finish the vector leg** once API paths land, and run the genuine three-way hybrid. 5. **Neptune full-text search** to replace the substring scan. 6. **Parallelize extraction + batch Gremlin writes** — together the biggest wall-clock win. 7. **Per-triple provenance** — prerequisite for trusting a graph-derived answer in a service context.

8. Q&A: LangGraph, LLM engineering & knowledge graphs

Q29. Why LangGraph and not CrewAI or AutoGen?

"Because I know the pipeline, and autonomy would be a liability rather than a feature. Ingest is fixed ETL with probabilistic steps; query is a router with three known corrective branches. LangGraph gives explicit typed state, deterministic control flow, conditional edges, and step-level observability. A crew would add nondeterminism on a path where **the ontology is the contract**, and I'd lose reproducibility. The framing I'd use: **the orchestration is deterministic; only the nodes are probabilistic.**"

Q30. What does LangGraph actually give you over plain Python functions?

"I'll concede the counter first: for a 200-line prototype, plain functions would have been fine. What I get is four things. **Typed shared state** via `TypedDict`, so I'm not threading a dict through eight call sites. **Conditional edges**, which are a first-class expression of my core design — `_route_after_eval` is a one-line function returning the verdict label, and the edge map turns that into a branch, so the policy is *declared* rather than buried in if-statements. **A free trace**, so every query returns which branch fired and what it cost. And **a path to checkpointing** — adding a `MemorySaver` means a late failure resumes without re-paying for the expensive calls already made. That last one is the deployment-readiness argument, and deployment was the stated goal."

Q31. Describe your state machines precisely.

"**Agent 1:** `assess → (conditional) → store_graph | store_vector → write_catalog → END`. The conditional edge is the router; the condition is the entity gate.

Agent 2: `route → retrieve → evaluate → (conditional) → correct | ambiguous | incorrect → generate → END`. The condition function returns the verdict label and the edge map routes it.

Same pattern in both: the interesting logic is **module-level pure functions** — `evaluate`, `refine`, `generate`, `assess_document` — and the `_n_*` nodes are thin wrappers that update state and increment the call counter. So the logic is unit-testable without constructing a graph, and the StateGraph is wiring."

Q32. You hit a LangGraph problem. What was it?

Tell §6.7. "Cross-branch state merges after a conditional edge were unreliable, so propagating `in_graph` / `in_vector` as node state from the two branch nodes was fragile — a value set in one branch wasn't dependably visible after the merge. Rather than fight the framework, I compute the flags **deterministically** in `write_catalog` from `build_graph` plus the dual-store config, and documented it so nobody 'fixes' it back into the fragile form. General principle: **when a framework's guarantee is unclear, derive the value from inputs you control.**"

Q33. How do you get reliable structured output from an LLM?

"Layered, because any single layer fails. **Request** JSON mode via `response_format`. **Specify the exact shape** in the system prompt, including casing conventions — PascalCase types, UPPER_SNAKE_CASE relations — because a stated convention is followed far more often than an inferred one. **Parse tolerantly** — a fenced ````json` block still happens, so strip fences before raising, and raise *with the content* so the failure is diagnosable. **Validate structurally** and drop what doesn't conform: malformed triples, empty fields, self-loops. And **never let one bad response be fatal** — a window whose JSON won't parse is skipped and logged, because one bad response must not lose an entire manual."

Q34. How do you control LLM cost?

"Structurally, not by cutting quality checks — that's the principle I'd lead with. Five levers. **Model tiering**: two configured deployments and a `cheap` flag set at every call site; `generate` is the *only* place passing `cheap=False`. **Early exit**: a good grade ends the query. **Conditional execution**: refine only above 180 words; grading skipped entirely on empty retrieval. **Bounded fan-out**: hard caps on scan and extraction windows, with even sampling above the cap so coverage stays whole-document. **Push work left**: the entity scan is paid once at ingest and amortized across every query it helps route.

Net: a typical query is **2–4 calls with exactly one on the strong model**. And note the framing — I never dropped a quality call to save money; I changed the tier, the condition, or the cap."

Q35. What's in your prompts that a beginner wouldn't put there?

"Six things, and I can point at each. **Explicit output shape**, not a description of it. **Canonical-naming instructions** — 'Error Code E-204', not 'the E-204 code' — because entity merging is by name, so the prompt is doing data-modelling work. **Casing conventions**, so downstream label sanitization is predictable. **An explicit negative constraint** — 'only include relationships the text explicitly supports, do NOT invent facts.' **Explicit furniture exclusions** — page numbers, headers/footers, bare headings, generic filler — because real PDF text is full of it and it silently poisons extraction. And **permission to return nothing** — 'return an empty list if the text asserts no concrete relationships' — because without that, a model asked for triples will manufacture them."

Q36. How do you handle a document too long for the context window?

"Windowing with bounded, **evenly-sampled** coverage — the `_windows` function in §3.4. Rather than truncating to the first 8 windows, which would assess only the front of a long manual, it spreads 8 sample points evenly from start to end. Entities are normalized and unioned across windows, so windowing doesn't cost me deduplication.

Two honest limitations: PageIndex extraction caps input at ~6000 words, so a very long document's outline is built from a truncated view; and even sampling means I genuinely might miss a window's content. Both are cost guards that **silently reduce coverage**, and my position is they should **log what was dropped** — silent truncation reads as complete coverage when it isn't."

Q37. Temperature 0 doesn't work on reasoning models. How do you get reproducibility?

"You don't get it from sampling, so you engineer it elsewhere. **Deterministic IDs** from entity names, so re-running converges on the same nodes even if wording varies. **Idempotent upserts**, so re-running produces the same graph rather than accumulating drift. **Structural validation**, so out-of-contract output is rejected rather than persisted. **Majority-vote typing**, so one anomalous run doesn't change a node's label. And **deterministic parsing for anything with a correct answer** — the `Document → Section → Chunk` spine is regex-parsed precisely because structure shouldn't vary between runs. The mindset shift: **reproducibility becomes a property of the assembly rules, not of the model call**."

Q38. How do you test something whose core component is nondeterministic?

"Separate the orchestration from the model, then test the orchestration exhaustively. All model access is behind a **two-method interface** (`complete_text`, `complete_json`), and the fake **inspects the system prompt** to know which pipeline step is calling it. The key choice: the fake's evaluator score is a **deterministic term-overlap recall** between question and context — so a verdict is a *consequence* of retrieval quality rather than a hard-coded value. If routing broke, the verdict would change and a test would fail.

That's how 11 tests cover catalog round-trip, both routing paths, all four Agent 2 outcomes, batch ingest, document scoping, and the dual-store toggle in 0.7 seconds. What it does **not** test is prompt quality or real model behaviour — and I'd say so unprompted. Offline tests prove the machine is wired correctly; they say nothing about whether the model is any good."

Q39. Your fakes sound suspiciously convenient. Aren't you testing your own assumptions?

"Partly, and that's the honest limit of any test double. Two things keep it useful. The fakes are **logic-driven, not scripted** — see above. And they're **adversarially designed**: `FakeDIS` stores only the first 60 words of each document, so deep-section queries genuinely miss the vector store and must be recovered by vectorless navigation. **I built the fake to make my system fail in the way I claimed it could fail**. What they can't tell me is whether the real model grades the way the fake does — the docs note verdicts skew, e.g. grading 'correct' where a real model would say 'ambiguous'. Known gap, not a surprise."

Q40. Walk me through how the LLM builds the graph.

Reference §3.7. "Windowed triple extraction on the **strong** deployment — subject/relation/object with typed endpoints, strict JSON, PascalCase types and UPPER_SNAKE_CASE relations that the model chooses. Then five assembly rules in code, which is the 'LLM proposes, code enforces' part: entities **merge by name-derived id** across windows and documents; each node's type is decided by **majority vote** across the mentions that produced it, so one bad window can't mislabel a node; **duplicate triples increment an edge count** rather than creating parallel edges, so repetition becomes a confidence signal; self-loops from name collisions are dropped; and a window whose JSON fails to parse is skipped, not fatal."

Q41. Your LLM assigns node labels. Isn't that a schema nightmare?

"It's a real risk and I'd name it before you do. Within an entity, majority vote keeps typing stable. Across a **corpus**, nothing stops drift — `Component` vs `Hardware` vs `Part` for the same concept. At three documents that's invisible; at three thousand it's a mess.

Two mitigations. Design-wise, a **controlled vocabulary**: infer types on a sample, freeze the list, constrain later extraction to it — which is the pattern I used in the sibling PDF agent, where the LLM proposes **one shared ontology per batch** and a validation node rejects non-conforming instances. Storage-wise, because Neptune labels are immutable, the type also lives in an `etype` property and **all reads filter on that** — so re-typing is a property update, not a rebuild."

Q42. Why is the structural spine deterministic if the LLM is so good at this?

"Because document structure is a **parsing problem with a correct answer**, and relationship extraction is a **judgement problem**. Spending LLM calls on the first adds cost and variance for nothing — and worse, it makes the section tree non-reproducible, which matters because the tree is what vectorless navigation reads. So I spend model calls only where there's genuine ambiguity. Note I do *also* have an LLM PageIndex extractor, and the interesting finding is that on clean markdown the LLM tree and the parsed tree **largely agree** — I read that convergence as a correctness signal. The LLM path earns its keep on messy sources where a parser degrades badly."

Q43. How do you handle entity resolution / deduplication?

"Today: name-derived canonical IDs, so exact name matches merge across sections and documents; majority-vote typing; duplicate triples incrementing a count. What that **doesn't** solve is aliases — 'Detector' and 'the Detector assembly' stay separate nodes, which is my main source of fragmentation. Ranked fixes: **canonical-name carry-forward** into later windows (cheapest, attacks the root cause, it's a prompt change), then an **alias-merging post-pass** using substring/fuzzy/embedding similarity, then a proper entity-resolution model only if it ever justified the complexity. In that order, because the first is a prompt change and the last is a project."

Q44. Why is graph fragmentation "expected rather than a defect"?

"Because of how the two layers are built. The spine is connected **by construction** — every section hangs off one [Document](#) node. Two extracted triples only connect if they **share an entity name**, and windows are extracted **independently**, so the same real thing can surface with two different surface forms and fail to merge. That's a mechanical consequence of windowed independent extraction, not a bug in the assembly. Which is also why the fix is at the extraction prompt — carry names forward — rather than in the graph builder."

9. Q&A: Neptune, AWS, auth, security, testing & scale

Q45. Walk me through the authentication flow.

Draw it, don't just say it: `LEG 1 POST /oauth2/token grant_type=client_credentials client_id + client_secret + scope → access_token` `LEG 2 POST /oauth2/token grant_type=token_exchange HTTP Basic(client_id:client_secret) token=<access_token> + audience=<lmaas> → exchange_token` ↓ Azure OpenAI SDK: `azure_ad_token_provider` "Two legs. The plain client-credentials token is **rejected** by the gateway; it must be exchanged for a scoped token — RFC 8693, with the client credentials as HTTP **Basic** auth on the second leg. The exchange token is what the SDK carries, and I hand the SDK the generator **as a callable** via `azure_ad_token_provider`, not a token value, so every request picks up a fresh token automatically instead of me remembering to refresh.

The vector platform uses the same shape with a different `audience`, and its token additionally carries a **mapped RBAC roles claim**. That's why there are two generator classes rather than one with a flag: they return tokens with different *semantics*, not just different values. It also lets me map agents to least privilege — Agent 1 needs Admin to ingest, Agent 2 only needs User."

Q46. How do you manage the token lifecycle?

"Cached in-process, refreshed when within **60 seconds of expiry**, behind a **double-checked lock** — fast path outside the lock, re-check inside it. With parallel extraction running multiple threads, several threads hitting an expired token simultaneously would otherwise stampede the token endpoint with identical exchanges. `time.monotonic()` rather than `time.time()`, so a clock adjustment can't invalidate a valid token. And the generator is **injected** into the client, so tests pass a stub and never touch the network."

Q47. You had a 403 that wasn't auth. Walk me through the debugging.

Tell §6.3 as a story: symptom → the `Server: awselb/2.0` header → three endpoint variants → wrong one from AWS → PrivateLink fix. Land the diagnostic: **"401 means we don't know you; 403 with `awselb`, including on a request with no auth header at all, means your network isn't allowed — a real auth layer would have said 401."** Then the positive check: decode the token, verify the `aud` claim. "Those two tests together are two-sided: `aud` correct proves identity, unauthenticated-403 proves network. Between them I can always tell which layer is failing."

Q48. Explain PrivateLink and why you needed it.

"The gateway publishes a **VPC endpoint service**. I create an **interface VPC endpoint** from it in the notebook's VPC — private ENIs — and the **Internal VPCE DNS** name resolves to those, so traffic reaches the service over the AWS network without touching the internet. Three reasons that matters: no egress in a locked-down VPC, no data leaving the AWS backbone, and the External hostname is IP-allowlisted to the corporate network so it simply won't work from AWS.

And it's **orthogonal to Neptune reachability** — Neptune needs the notebook to be *in* or peered to the Neptune VPC with a security group allowing TCP 8182. Two different network paths, two different failure modes, and conflating them cost me time."

Q49. How do you authenticate to Neptune?

"IAM database authentication — every Gremlin request is **SigV4-signed** for service `neptune-db` using the notebook role's boto3 credentials. **No database passwords anywhere**: access is IAM policy, so it's auditable in CloudTrail and revocable

centrally. One practical consequence — signing is per-request work, so on a write-heavy path connection pooling saves not just TCP+TLS setup but **re-signing overhead** on every call."

Q50. How do you handle secrets?

"Env-only, never in code. Every secret and endpoint defaults to the literal `REPLACE_ME`, and `is_configured()` returns false for empty-or-`REPLACE_ME`. The real clients call `_require(...)` with named pairs and **fail fast with a message listing exactly which settings are unset**, instead of surfacing a confusing 401 from a service. Small thing, saves a lot of debugging — an unconfigured system should tell you what's missing, not what broke. Production injects via Secrets Manager or Parameter Store; the code doesn't change because it only ever reads env vars."

Q51. What are the security risks in this system?

Answer in threat-model order — this is where you sound senior: 1. **Prompt injection** → **query injection**. LLM-extracted names and relation labels are **inlined into Gremlin scripts**, because Neptune's HTTP endpoint doesn't support bindings, and a document is attacker-influenceable content. Mitigated by `_lit()` escaping and `_label()` whitelisting to `[A-Za-z0-9_]`. First thing I'd flag in a review. 2. **Prompt injection** → **generation**. A malicious document can carry instructions to the generator. **Not currently mitigated**. I'd add delimiter discipline, an instruction-hierarchy prompt, and output checks. 3. **Data sensitivity**. The gateway is **non-production, no PHI/PII** — enforced by using synthetic documents throughout. Highest-severity failure available in this project, so it's the constraint I'd state before any technical detail. 4. **Over-broad graph access**. Anything that can query the graph sees everything. Real multi-tenancy needs per-collection authorization; the vector platform has RBAC and I map agents to **least privilege**. 5. **XSS in generated HTML**. Manual text goes into a visualization page, so angle brackets are escaped. 6. **TLS verification disabled** for internal CAs. Made **config with an explanatory comment** rather than a silent default — "the sample did it" isn't a security argument.

Q52. Why route model calls through an internal gateway instead of a provider directly?

"Governance, and it's the right call even where it's inconvenient. Centralized auth via IDAM with short-lived tokens instead of long-lived keys; an audit trail of who called which model with what; data residency — nothing leaves the VPC; plus cost attribution and centrally-managed deployments. The cost is that I don't choose my own models — I get what's provisioned, which is exactly why fine-tuning and a custom reranker weren't options. In regulated healthcare that trade is straightforwardly worth it, and it's far easier to defend in a security review than 'we call an external API'."

Q53. Tell me about a Gremlin query you're proud of, or one that was hard.

"The **edge upsert**, because it encodes two Neptune facts:

```
groovy g.V(source).as('s').V(target).coalesce( __.inE(REL).where(__.outV().hasId(source)),
__.addE(REL).from('s')).property('count', n)
```

It **materializes the source endpoint as a step label** `as('s')` and checks for an existing `(source, relation, target)` edge before adding. Two payoffs: it's idempotent on the triple, and if an endpoint is missing the statement is a **safe no-op** instead of an error — so dangling edges from imperfect extraction never fail a run. I learned that pattern the hard way in a sibling project, where the natural form `addE(label).to(__.V(target)).property(id, eid)` was rejected with `ConstraintViolationException`: with a **custom edge id**, Neptune's planner can't statically prove an anonymous `__.V(target)` resolves to exactly one vertex, so it refuses to guarantee uniqueness. Pinning endpoints to step labels first is what lets it."

Q54. What broke when you first read data back from a real cluster?

Tell §6.4 — the untyped-GraphSON header. "Reads returned **empty results with no error**. Neptune returns typed GraphSON v3 by default, so a list arrives as `{"@type": "g:List", "@value": [...]}` and my read helpers received a dict where they wanted a list. A *shape* bug, not a failure — the worst kind, because nothing raises. One header fixed it: `Accept: application/vnd.gremlin-v3.0+json;types=false`.

Two of this project's failures produced **silently wrong** results rather than errors — this one and the threshold. Both taught the same habit: **for any integration, assert on the parsed shape early**, because a 200 with an unexpected structure will quietly propagate emptiness through five layers."

Q55. How is this tested?

"11 offline tests, 0.70 seconds, zero credentials — and I ran them to confirm before this conversation. They cover the catalog round-trip, both Agent 1 routing paths, the full-document entity gate, the dual-store toggle, all four Agent 2 outcomes, batch ingest, and multi-document scoping. The enabling design is dependency injection plus fakes that duck-type the real clients — the public API is **identical fake-vs-real**, so swapping backends is just which objects you pass to the constructor.

Two details I'd point at: the fixture is engineered so a section **heading carries terms its body doesn't**, which is what makes all three CRAG branches reachable from one document. And the assertions check **recovered content**, not just branch identity — `assert "duty lead" in res.answer.lower()` proves vectorless recovered from the *right* document, not merely that it recovered something."

Q56. What's the cost per query, and where does it go?

"**2–4 LLM calls per query, exactly one on the strong model**. Correct path: 1 cheap grade + 0–1 cheap refine + 1 strong generate. Ambiguous: add a cheap navigation call. Incorrect: grading is skipped when retrieval is empty, so it can actually be *cheaper* — nav plus generate. Abstention costs no strong call at all.

Ingest is where the money is: ≤8 cheap entity-scan, 1 cheap domain, ≤6 strong triple-extraction, 1 strong PageIndex per document — but it's one-off per document and **amortized** across every query it serves. What I don't have is per-request token metering, so I can quote call counts confidently and dollar costs only roughly. I'd close that with per-call token accounting before anyone asked me to forecast spend."

Q57. Where's your latency, and what would you fix first?

"Three sources. **Reasoning-class deployments are markedly slower per call** — that's the floor and I don't control it. **Extraction is sequential per window** — the sibling PDF agent parallelizes it with a bounded thread pool, so I know the fix and didn't carry it over. **Gremlin writes are one HTTP POST per statement**, so a document with many sections takes a while to persist.

First fix: **parallelize extraction and batch the writes** — together the biggest wall-clock win, and both are already-proven patterns rather than new work. Batching has a specific trap worth naming: **every statement must end in `.iterate()`**, because Neptune only iterates the last statement of a submitted script — earlier ones are built as lazy traversals and silently discarded while the call still returns 200. On the query path the levers are early exit (already there), streaming generation for perceived latency, parallel fan-out on the ambiguous branch so retrieval is `max()` not `sum()`, and LRU caching on query→answer and query→verdict."

Q58. What would break at 10,000 documents?

"Five things, roughly in the order they'd hurt. **The catalog** — a single JSON file with term-overlap scanning is $O(n)$ per query with no cross-process locking; that becomes DynamoDB with a real index. **Ingest cost** — bounded per document but linear in documents, and strong-model triple extraction dominates; needs a cheaper tier and incremental re-ingest on revision only.

Write latency — one POST per statement; batching plus the bulk loader for the initial cold load. **Graph fragmentation** — gets worse with scale, because more documents mean more alias variants, so entity resolution stops being a nice-to-have. **Vectorless navigation** — the table of contents stops fitting in a prompt; already partly handled by scoping to catalog candidates, but at real scale it needs hierarchical two-stage navigation: pick documents, then pick sections."

Q59. What's genuinely production-ready and what isn't?

Be crisp and volunteer the negatives — this is a credibility test:

Area	Status
LMaaS integration (two-leg auth, endpoints, reasoning params)	✅ Working end-to-end from SageMaker over PrivateLink
Both agents' orchestration (LangGraph)	✅ Complete, 11 offline tests passing
LLM-built typed knowledge graph	✅ Working, persisted to Neptune ; connectivity improvable
LLM-built PageIndex tree	✅ Working
Neptune graph leg	✅ Live for write/read/visualize/vectorless — ⚠️ keyword search is still a substring scan; needs full-text search
Vector leg	⚠️ Client implemented against the documented contract; exact API paths pending ; demo uses an in-memory stand-in
Graph-native multi-hop retrieval	❌ Graph is persisted but not traversed at query time
Evaluation / gold standard	❌ Doesn't exist — extraction quality is eyeballed
Scale	⚠️ Prototype: three synthetic documents
Observability / cost metering	⚠️ Per-query trace only; no distributed tracing, no token accounting
Checkpointing	❌ A late failure re-pays every earlier LLM call
Data sensitivity	🚫 Non-production gateway — no PHI/PII

Q60. If this had to serve real users next month, what's the critical path?

"In this order. **Finish the vector leg** — a two-of-three hybrid isn't the design. **Build the eval set** — I won't ship thresholds I can't defend. **Batch writes and parallelize extraction** — ingest latency is a usability problem at real corpus size. **Full-text search on Neptune**, so the graph leg stops being a substring scan. **DynamoDB catalog** — the JSON file races under concurrency, which is a correctness bug the moment there are two ingest workers. **Prompt-injection hardening on the generator**, and **per-triple provenance** so answers are citable.

And **zero: the compliance conversation** — the gateway is non-production and real service manuals may not be PHI-free. That's a gate, not a task, and it comes before any of the engineering."

Q61. How would you monitor this in production?

"Four groups. **Quality**: verdict distribution over time — a sudden shift toward `incorrect` means retrieval or a store regressed; abstention rate; sampled human review of answers *with their traces*. **Cost**: calls and tokens per query split by tier, alerting on drift — a prompt change that quietly doubles strong-model calls should page someone. **Latency**: p50/p95 **per node**, not per request, so I know which stage regressed. **Correctness**: extraction precision/recall against the gold set on every prompt change, plus citation-accuracy sampling. And retain the per-query trace — it already exists, and it's what makes a user complaint diagnosable rather than a mystery."

Q62. How would you add multi-hop graph retrieval — the thing you say is missing?

"It's a fourth retrieval mode, and the pieces are already in place. The typed graph is persisted with `etype` on every vertex and the relation as the edge label, so a traversal is expressible today. Three steps. **Anchor:** resolve query mentions to entity nodes — exact match on `name` first, then fuzzy, which is also the entity-anchored routing upgrade. **Traverse:** bounded-depth expansion from those anchors, following typed edges and ranking paths by edge `count` — which is why I recorded `count` as a confidence signal rather than deduplicating silently. **Serialize:** turn the resulting subgraph into text the generator can read, which is its own design question — path sentences ('Error Code E-204 INDICATES Collimator Alignment Fault, RESOLVED_BY Collimator Alignment Routine') plus the source chunk text via `MENTIONS`.

Then it slots into `_retrieve` as a third source and the grader treats it identically, because every retriever returns the same `RetrievalResult`. The blocker isn't the traversal — it's that without an eval set I couldn't tell whether it beat vectorless."

10. Behavioral / STAR stories

Four stories, mapped to the questions they answer. Practice each at **90 seconds**.

10.1 "Tell me about feedback you received" → the meaningless threshold

S/T: My graph-worthiness gate counted entities from a curated sample of the first 2000 characters, capped at five, threshold of three. It worked and I'd moved on. **A:** A reviewer pointed out the number 3 was measuring **sample size, not the document** — a 200-page manual and a 2-page memo could score identically, and no value of that threshold would ever mean what a reader assumes. They were right and I hadn't seen it. I rebuilt the gate: whole-document 400-word windows, one cheap call each, entities normalized and de-duplicated across windows, count **distinct**, threshold 8 — with a hard cap of 8 windows and **even sampling** above it, so coverage stays whole-document while cost stays bounded. **R:** The threshold now means exactly what it says. Bonus: the scanned entities enrich the catalog and improve query-time routing, so the extra work pays for itself even for vector-only documents. **Lesson:** a magic number is a bug when it doesn't correspond to the thing it claims to measure — and I'd rather be corrected early than defend a number I can't explain.

10.2 "Tell me about a difficult bug" → debugging the wrong component

S/T: Vectorless navigation was choosing irrelevant sections. The obvious move was to tune the navigator prompt. **A:** Before that, I printed what the navigator was actually reading — and the **table of contents was garbage**. My heading detector treated single-number lines (**1 Do X**) and short ALL-CAPS lines as headings, so on real manual text every numbered procedure step and every UI label had become a section. The model was choosing correctly from noise. I narrowed the detector to two strong signals only — markdown **#** or dotted multi-level numbering — which created a follow-on problem: real PDFs have no **#**. So I moved heading detection into the **PDF loader, by typography**: derive the modal body font size, then treat a line as a heading if it's $\geq 15\%$ larger, or bold and short, and emit **#**. **R:** Navigation quality fixed at the source, and the design got better rather than patched: structure detection now lives where the font information is, and the section parser is a two-condition function. **Lesson: debug the input, not the component.** I nearly spent a day prompt-tuning a component that was behaving correctly.

10.3 "Tell me about being blocked" → building against an unavailable dependency

S/T: The managed vector platform's exact API paths and response shapes lived in an API collection I didn't have access to yet, and gateway credentials were pending. The instinct was to wait. **A:** Instead I made the blocker **structural rather than temporal**. Every backend went behind a small interface; both agents take (**lmaas, dis, graph**) as constructor arguments. I wrote fakes that duck-type the real clients — and *designed* rather than stubbed them: the fake LLM inspects the system prompt to know which step is calling it, its evaluator score is a real term-overlap recall so verdicts follow retrieval quality, and the fake vector store **deliberately indexes only each document's first 60 words** so the corrective path is genuinely exercised. Real clients were implemented against the documented contract with explicit **REPLACE_ME** constants and fail-fast config checks. **R:** The entire two-agent design, every CRAG branch, routing, scoping, and fusion were built and validated — 11 tests in 0.7 seconds — before a single credential existed. When gateway access landed it was a constructor swap, and the same code paths served the live backend. **Lesson:** an external blocker is usually a design problem in disguise.

10.4 "Tell me about a technical decision you'd defend" → no web search on the corrective path

S/T: Canonical Corrective RAG falls back to **web search** when retrieval grades poorly. That's the published design and the obvious thing to implement. **A:** I couldn't — no VPC egress — but I argued I *shouldn't* anyway: for internal medical-device service documentation, answering from the open web is a **worse** failure than abstaining, because it produces a confident, uncited answer about equipment safety from a source nobody approved. So I replaced the corrective *source* while keeping the CRAG *mechanism*: vectorless navigation over our own document tree, with a fallback to the other internal store, and **explicit abstention** if even that is empty. **R:** Fully internal, no egress, and better suited to the failure mode I actually had — embeddings under-representing deep procedural sections. It also cost nothing extra to build, because the section tree was already a by-product of graph ingest. **Lesson:** implementing a paper faithfully isn't the goal — implementing its *insight* under your real constraints is.

11. Killer questions

The ones designed to find your limits. Have these rehearsed.

"Your typed graph isn't even used at query time. So why build it?"

"Fair hit, and it's the limitation I lead with rather than wait to be caught on. Today the graph leg is keyword search over chunk text; the typed entity graph is persisted but not traversed. What it *does* already earn: the structural spine it's built on powers the entire vectorless corrective route — the mechanism that recovers both failure branches — and the routing catalog is populated from extracted entities, which is what makes query routing non-blind. So the ingest work isn't idle. But the biggest unclaimed payoff in the system is multi-hop traversal, I'd rank it as the number-one next feature, and I can tell you exactly how it slots in as a fourth retrieval mode."

"This looks like a lot of complexity for three synthetic documents."

"It is, and that's the correct read of a prototype. What I'd push back on is the implication that the complexity is speculative — every branch exists because a **specific named failure mode** exists, and each is reproducibly demonstrated in the test suite: a heading-only match where body search returns nothing, a partial match that must fuse, and an out-of-corpus question that must abstain. The architecture is validated on three documents; the *quality* isn't validated at all, and I'd say so. And I can tell you what to cut if it had to be simpler: drop the entity graph, keep the section tree, and you have a leaner vector+vectorless system that keeps the self-correction and loses relational retrieval."

"How do you know the LLM extraction is any good?"

"I don't, numerically, and it's the biggest gap in the project. No gold-standard triple set, so precision and recall are **eyeballed, not measured** — meaning I can't even tell whether a prompt change helped. What I do have is defensive assembly: majority-vote typing so one bad window can't mislabel a node, duplicate triples incrementing a count rather than creating parallel edges, malformed and self-loop triples dropped, and a per-window failure skipped rather than fatal. That's damage control, not evaluation. If you asked for one artifact before any further tuning, it's a labeled triple set — I'd build it before touching another prompt."

"Why should I believe the grader isn't just adding cost and latency?"

"Because its cheapest branch is free and its expensive branch only fires when quality is at risk. Empty retrieval short-circuits to **incorrect** with **zero** LLM calls. A good grade **early-exits** — so on the common path the grader costs one cheap call and saves a wasted navigation call plus a wasted strong-model generation over bad context. That's the trade: one cheap call per query to avoid confidently answering from irrelevant context. What I *can't* prove without an eval set is the false-**correct** rate, which is the number that would actually settle it — and that's the honest boundary of my claim."

"What's the one thing you'd change about your own design?"

"Build the evaluation harness first. Everything I'm least confident about — the two thresholds, extraction quality, whether the graph leg earns its keep, whether the corrective branch fires as often as I think — is unmeasurable without it, which means every improvement past this point is a guess dressed up as engineering judgement. I got the architecture right by reasoning about failure modes; I can't get the *tuning* right the same way."

"Convince me you understand this deeply rather than having assembled it from tutorials."

"Ask me about the failures rather than the features. Neptune returns typed GraphSON by default, so my reads came back silently empty until I asked for **types=false**. A custom edge id makes Neptune refuse an anonymous **__.V(target)** endpoint, so upserts need materialized **as('s')** step labels. Neptune vertex labels are immutable, so an LLM-assigned type has to live in an **etype** property too. The gateway's 403 carried **Server: awselb/2.0** — a load-balancer block, not an auth failure — and an unauthenticated request *still* returning 403 instead of 401 is how you prove that. Auth is a two-leg token

exchange whose first leg produces a token the gateway rejects. Reasoning deployments reject `max_tokens` and any non-default `temperature`, so you can't even set `temperature=0`. And LangGraph's cross-branch state merge was unreliable enough that I compute my routing flags deterministically instead. None of that is in a tutorial. It's all in my decision log, with the reasoning, so the next person doesn't pay for it twice."

"You keep saying 'the grader catches it.' What if the grader is the thing that's wrong?"

"Then I have a silent failure, and I'll name the two directions. A false `incorrect` is cheap — one extra navigation call, and fusion or fallback usually recovers. A false `correct` is the dangerous one: it skips correction entirely and generates over bad context. That asymmetry is exactly why the correct threshold is 0.7 rather than 0.5 — I've biased toward over-correcting. The unmitigated risk is systematic grader bias, which no threshold fixes, and the instrument for that is grader-vs-human agreement measurement. I don't have it. The one mitigation I *did* ship is that a grader **error** defaults to `ambiguous`, so a crash widens retrieval rather than trusting it."

11.1 Questions to ask them

Pick three or four; they signal seniority as much as your answers do.

On the work: 1. "Where does retrieval quality break down for you today — is it recall, precision, or the fact that nobody knows when it broke?" 2. "Do you have an evaluation set, and is it trusted? That's the thing I'd want on day one." 3. "How do you handle document revisions? Superseded content is the problem I could least convincingly solve — idempotent writes update in place, but nothing deletes what a new revision removed." 4. "Is anyone doing graph-native multi-hop retrieval here, or is the graph mostly a store today?"

On engineering culture: 5. "How do prompt changes ship? Is there a gate, or is it code review plus hope?" 6. "What's the split between prototype work and production hardening on this team?" 7. "How much say does an engineer have in model choice, and does the governed-gateway path constrain that?"

On constraints: 8. "What's the PHI/PII posture for this workload — does it ever touch production data, and what changes when it does?" 9. "How do you think about cost per query? Is it metered per request, or budgeted at the platform level?"

12. The 20-minute pre-interview skim

Memorize verbatim

- The **30-second pitch** (§1.2).
- "401 means we don't know you; 403 with `awselb` means your network isn't allowed."
- "Never cut a quality call — cut the model tier, the condition, or the wait."
- "The orchestration is deterministic; only the nodes are probabilistic."
- "Fusion, not selection."
- "The LLM proposes; code enforces the contract."
- "Reproducibility is a property of the assembly rules, not of the model call."
- "Debug the input, not the component."

Numbers

0.7 / 0.3 verdict bands · 8 distinct entities to earn a graph · 400-word windows, 8 max · 180 words before refining · top_k=5 · 120-word chunks · 4000-char grader context · ≤4 sections per navigation · 60-second token skew · 2–4 LLM calls/query, one strong · 11 tests, 0.70 s, no credentials · ~2,500 lines, 12 modules

The four war stories

1. **The threshold that measured sample size** (§6.1) → "a magic number is a bug when it doesn't measure what it claims."
2. **The heading detector that flooded the tree** (§6.2) → "debug the input, not the component."
3. **The `awselb` 403** (§6.3) → the 401-vs-403 diagnostic.
4. **Typed GraphSON returning silently empty reads** (§6.4) → "assert on the parsed shape early."

The five "why not X" you'll definitely be asked

1. **Why not Neo4j?** → Platform/VPC/IAM; no new vendor in regulated healthcare. "Greenfield, I'd pick Neo4j."
2. **Why not plain vector RAG?** → The four failure modes: joins, silent failure, not every doc deserves a graph, uneven coverage.
3. **Why not Microsoft GraphRAG?** → Its indexing pass is expensive and it optimizes *global* sensemaking; my questions are local and specific.
4. **Why not CrewAI/AutoGen?** → I know the pipeline; autonomy is a liability where the ontology is the contract.
5. **Why no web-search corrective?** → No egress, *and* answering from the open web about device safety is worse than abstaining.

Volunteer these three limitations before you're asked

1. The typed graph is **persisted but not traversed** at query time.
2. There's **no gold standard** — extraction quality is eyeballed, so thresholds are reasoned, not fitted.
3. The vector leg runs an **in-memory stand-in**; exact API paths were pending.

Volunteering limitations is not a weakness in an interview. It's the strongest available signal that you know the system better than the person evaluating it.

If you only have five minutes

Read §1.2 (the pitch), §2.5 (the thresholds table), and the four war-story headlines above. Then draw the two agent diagrams from §2.2 on paper once, out loud.